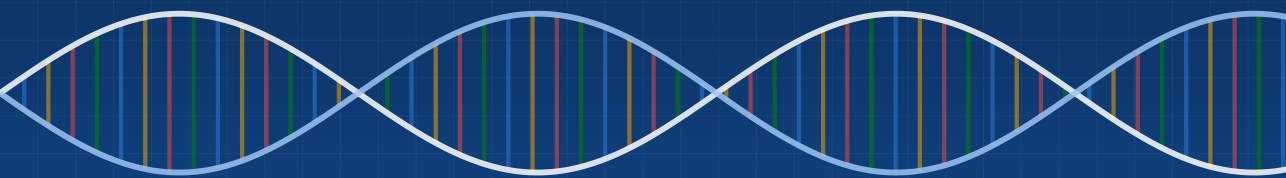


Flujos reproducibles y proyecto final



Un artículo sobre ciencia computacional en una revista científica no es la investigación misma: es solo la publicidad de la investigación. La investigación de verdad es el entorno completo de desarrollo de software y el conjunto completo de instrucciones que generaron las figuras.

— Buckheit y Donoho (1995), a propósito de J. Claeubout

● Al terminar este capítulo usted podrá

- Representar un análisis bioinformático como un grafo acíclico dirigido de trabajos, derivar el algoritmo de Kahn para ordenarlo y demostrar que detecta ciclos en tiempo $O(|V| + |E|)$.
- Escribir un flujo completo en Snakemake (reglas, comodines, inferencia hacia atrás) y su equivalente en Nextflow (procesos y canales), y explicar la diferencia entre un modelo *pull* y uno *push*.
- Fijar las cuatro capas de la reproducibilidad (código, entorno, datos y aleatoriedad) con Git, Bioconda, contenedores, sumas de verificación y semillas.

- Acotar el tiempo de ejecución de un flujo en paralelo con el trabajo y el camino crítico, y usar las leyes de Amdahl y Gustafson para decidir dónde vale la pena invertir núcleos.
- Diseñar, ejecutar, documentar y publicar un proyecto integrador de extremo a extremo que cumpla los principios FAIR y las obligaciones éticas y legales sobre datos humanos.

Hemos recorrido un largo camino. En el ?? aprendimos a fijar semillas, versiones y sumas de verificación; en los capítulos ?? a ?? convertimos lecturas crudas en alineamientos y estos en variantes anotadas; en el ?? reconstruimos árboles, y en el ?? pasamos de lecturas de ARN a genes diferencialmente expresados y a términos funcionales enriquecidos. Cada capítulo terminó con uno o varios bloques de código que funcionan *aislados*. Pero un análisis real no es una colección de bloques: es una cadena de decenas de pasos, aplicada a decenas o miles de muestras, que debe poder reanudarse cuando algo falla, repetirse cuando llega una muestra nueva y, sobre todo, ser reproducida por otra persona (o por nosotros mismos dentro de dos años) con exactamente el mismo resultado.

Este capítulo final trata de esa cadena. En la primera lección (sección 18.1) formalizamos un análisis como un grafo acíclico dirigido, derivamos el algoritmo que lo ordena y estudiamos los dos gestores de flujos más usados en bioinformática, Snakemake y Nextflow, junto con las herramientas que congelan el entorno de software (Bioconda y los contenedores) y la teoría que dice cuánto se gana al paralelizar. En la segunda (sección 18.2) aplicamos todo a un *proyecto integrador*: de las lecturas FASTQ de aislados de un patógeno a variantes anotadas y una filogenia (o, alternativamente, de un experimento de RNA-seq a un análisis de enriquecimiento), con su estructura de directorios, control de versiones, pruebas, informe reproducible, depósito de datos, consideraciones éticas y rúbrica de evaluación. La revisión de Wratten et al. (2021) y las “reglas” de Sandve et al. (2013) y Wilson et al. (2017) son buenas compañeras de lectura para todo el capítulo.

18.1 Snakemake y Nextflow

La cocina de un restaurante a la hora punta

Un cocinero que prepara un plato en casa sigue la receta de arriba abajo: pica, sofríe, hierve, sirve. En la cocina de un restaurante, a la hora punta, esa estrategia sería un desastre. Allí nadie piensa en la receta como una lista, sino como una red de dependencias: la salsa necesita el fondo, el fondo necesita los huesos tostados, el emplatado necesita la salsa y la guarnición, que no dependen entre sí y pueden prepararse a la vez en fogones distintos. El jefe de cocina hace tres cosas que un cocinero doméstico no hace: reparte en paralelo lo que es independiente, no repite lo que ya está listo en la cámara (*mise en place*) y, cuando un plato vuelve a la cocina porque la salsa salió mal, rehace la salsa y todo lo que depende de ella, pero no vuelve a tostar los huesos. Un gestor de flujos de trabajo es ese jefe de cocina para nuestros análisis.

18.1.1 Por qué los *scripts* no escalan

La forma más natural de automatizar un análisis es escribir un *script* de `bash` que llame a las herramientas en orden. Para una muestra y una tarde de trabajo, funciona. Pensemos, sin embargo, en el flujo que construimos a lo largo de los capítulos ?? a ??, aplicado a 24 aislados bacterianos: control de calidad con `fastp`, mapeo con BWA-MEM, marcado de duplicados, llamado conjunto de variantes con `bcftools`, filtrado, anotación, secuencias consenso y un árbol filogenético. Un *script* lineal tiene cuatro defectos que crecen con el tamaño del problema:

1. **No sabe qué está hecho.** Si el mapeo de la muestra 17 falla por falta de memoria, volver a lanzar el *script* repite el control de calidad y el mapeo de las 16 anteriores. Añadir la muestra 25 obliga a reprocesar las otras 24, o a editar el *script* a mano.
2. **No sabe qué es independiente.** Los 24 mapeos no dependen unos de otros y podrían ejecutarse a la vez, pero un bucle `for` los ejecuta en serie. Paralelizar a mano (con `&` y `wait`) es frágil y no respeta las dependencias.
3. **Mezcla la lógica con el entorno.** Las rutas absolutas, el número de núcleos o la versión de `samtools` que “haya en el sistema” quedan escondidos en el código, y el mismo *script* produce resultados distintos en otra máquina.
4. **Falla en silencio.** Sin `set -euo pipefail`, un error en medio de una tubería deja un archivo truncado que el paso siguiente consume sin quejarse.

Los gestores de flujos de trabajo resuelven los cuatro problemas con una misma idea: en lugar de describir *en qué orden* ejecutar los pasos, describimos *qué produce*

cada paso a partir de qué, y dejamos que un programa deduzca el orden, el paralelismo y lo que falta por hacer (Wratten et al., 2021).

18.1.2 Un análisis es un grafo acíclico dirigido

Definición 18.1. (Grafo de trabajos de un flujo)

Un *trabajo* (*job*) es la ejecución de un paso sobre entradas concretas: por ejemplo, “mapear la muestra A”. El *grafo de trabajos* de un flujo es el grafo dirigido $G = (V, E)$ cuyos vértices V son los trabajos y en el que existe una arista $(u, v) \in E$ si algún archivo producido por u es una entrada de v . Un *orden topológico* de G es una biyección $\pi : V \rightarrow \{1, \dots, |V|\}$ tal que

$$(u, v) \in E \implies \pi(u) < \pi(v). \tag{18.1}$$

DONDE

V, E	Conjunto de trabajos y conjunto de dependencias (aristas dirigidas del productor al consumidor).
(u, v)	Arista: v no puede empezar hasta que u termine.
$\pi(v)$	Posición de v en una ejecución en serie que respete todas las dependencias.
$ V , E $	Número de trabajos y de dependencias.

La figura 18.1 muestra el grafo de trabajos del flujo que usaremos como hilo conductor de todo el capítulo, con tres muestras (A, B y C). Tiene 19 trabajos y 26 aristas, y cada color corresponde a una *regla*, es decir, a una plantilla que se instancia una vez por muestra (*fastp*, *mapear*, *dedup*, *consenso*) o una sola vez para todas (*bwa_index*, *llamar*, *filtrar*, *anotar*, *filogenia*, *multiqc*). El último vértice, *all*, no ejecuta nada: solo declara qué archivos finales queremos.

¿Cuándo existe un orden topológico? La respuesta es el primer resultado que justifica todo el edificio.

Teorema 18.1. (Existencia del orden topológico)

Un grafo dirigido finito admite un orden topológico si y solo si no contiene ciclos dirigidos.

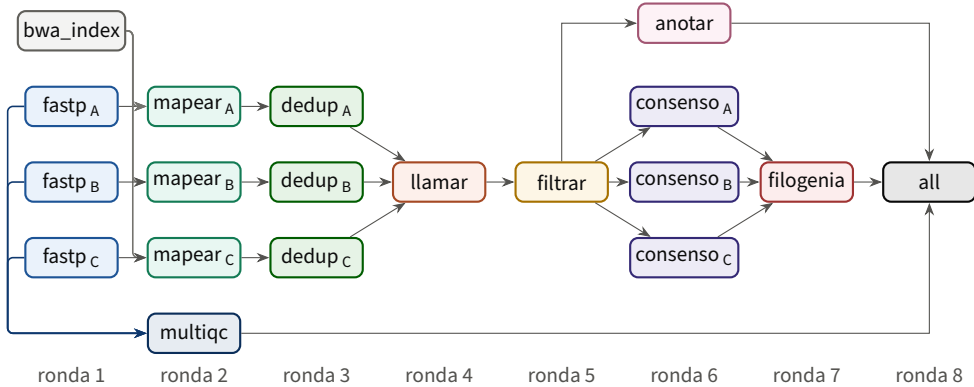


Figura 18.1. Grafo de trabajos del flujo de ejemplo con tres muestras (A, B, C): de lecturas crudas a variantes anotadas, secuencias consenso, un árbol filogenético y un informe de calidad. Cada color es una regla del Snakefile de la sección 18.1.5; los subíndices son los valores del comodín m . Las columnas son las *rondas* del algoritmo de Kahn: todos los trabajos de una ronda son independientes entre sí y pueden ejecutarse a la vez (anchos 4, 4, 3, 1, 1, 4, 1, 1). Observe los dos cuellos de botella de anchura 1, **llamar** y **filtrar**: el llamado conjunto necesita *todas* las muestras, y por ahí pasa todo el flujo. El índice de BWA (gris) se construye una sola vez y alimenta los tres mapeos; los tres informes de **fastp** confluyen en **multiqc** (azul oscuro, abajo). Grafo calculado con `figuras/cap18/generar.py`.

Demostración. ($\Rightarrow$) Si hubiera un ciclo $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$, la ecuación (18.1) exigiría $\pi(v_1) < \pi(v_2) < \dots < \pi(v_k) < \pi(v_1)$, lo cual es imposible. ($\Leftarrow$) Primero, un *lema*: todo grafo dirigido finito y acíclico no vacío tiene al menos una *fuentes*, un vértice sin aristas entrantes. En efecto, si todo vértice tuviera un predecesor, podríamos caminar hacia atrás indefinidamente, $v_0 \leftarrow v_1 \leftarrow v_2 \leftarrow \dots$; como solo hay $|V|$ vértices, tras $|V| + 1$ pasos repetiríamos alguno, y el tramo entre las dos visitas sería un ciclo. Ahora, por inducción sobre $|V|$: tomamos una fuente s , le asignamos $\pi(s) = 1$, la eliminamos junto con sus aristas (el grafo restante sigue siendo acíclico) y ordenamos el resto con las posiciones $2, \dots, |V|$. Ninguna arista entra en s , así que ninguna puede violar la ecuación (18.1). $\square$

La demostración es constructiva: *contiene* un algoritmo. Basta repetir “toma una fuente y elimínala”. El único problema es encontrar fuentes deprisa; recorrer todo el grafo en cada paso costaría $O(|V|^2)$. Kahn (1962), que buscaba ordenar las redes de actividades de los proyectos de ingeniería de su época (las redes PERT), resolvió el problema con una contabilidad sencilla.

18.1.3 El algoritmo de Kahn

Sea $d^-(v)$ el *grado de entrada* de v , el número de aristas que llegan a v . Eliminar una fuente s no cambia el grado de entrada de ningún vértice salvo el de sus sucesores, cada uno de los cuales pierde exactamente una unidad. Por lo tanto, las únicas fuentes *nuevas* que pueden aparecer al eliminar s son sucesores de s cuyo grado acaba de llegar a cero. Esta observación convierte la búsqueda global en una actualización local:

1. Calcular $d^-(v)$ para todo v recorriendo una vez todas las aristas. Poner en una cola Q todos los v con $d^-(v) = 0$.
2. Mientras Q no esté vacía: extraer v de Q , añadirlo al orden y, para cada sucesor u de v , hacer $d^-(u) \leftarrow d^-(u) - 1$; si $d^-(u)$ llega a 0, añadir u a Q .
3. Si al vaciarse Q el orden contiene menos de $|V|$ vértices, el grafo tiene un ciclo.

Teorema 18.2. (Corrección y coste del algoritmo de Kahn)

En todo momento, $d^-(v)$ es el número de predecesores de v que aún no están en el orden, y Q contiene exactamente los vértices no ordenados cuyos predecesores ya fueron ordenados. En consecuencia: (i) el orden producido satisface la ecuación (18.1); (ii) si G es acíclico, el algoritmo ordena los $|V|$ vértices; (iii) si termina con vértices sin ordenar, estos contienen un ciclo; y (iv) el tiempo total es

$$T_{\text{Kahn}} = O(|V| + |E|). \quad (18.2)$$

DONDE

$d^-(v)$	Grado de entrada “residual”: dependencias de v todavía pendientes.
Q	Cola de trabajos listos (todas sus dependencias satisfechas).
T_{Kahn}	Número de operaciones elementales del algoritmo.

Demostración. El invariante se cumple al inicio y cada decremento lo preserva, porque se hace justo cuando un predecesor entra en el orden. (i): un vértice entra en Q solo cuando todos sus predecesores ya fueron ordenados, luego queda detrás de todos ellos. (ii) y (iii): si Q se vacía con un conjunto $R \neq \emptyset$ de vértices sin ordenar, por el invariante ningún vértice de R tiene grado residual cero, es decir, el subgrafo inducido por R no tiene fuentes; por el lema de la demostración anterior, ese subgrafo contiene un ciclo. Si G es acíclico, esto no puede ocurrir. (iv): el cálculo

inicial de grados recorre cada arista una vez; cada vértice entra y sale de Q a lo sumo una vez, y cada arista se “consume” (se decrementa su destino) exactamente una vez, cuando sale su origen. $\square$

El punto (iii) es la razón por la que todos los gestores de flujos rechazan, antes de ejecutar nada, un flujo en el que una regla necesita (directa o indirectamente) su propia salida: la detección de ciclos sale gratis del ordenamiento.

Ejemplo 18.1. (Kahn sobre el flujo de tres muestras)

Los grados de entrada iniciales son $d^- = 0$ para `bwa_index` y los tres `fastp`; $d^- = 2$ para cada `mapear` (índice y lecturas limpias); $d^- = 1$ para cada `dedup`, para `filtrar`, `anotar` y cada `consenso`; y $d^- = 3$ para `llamar`, `multiqc`, `filogenia` y `all`. La suma es $4 \cdot 0 + 3 \cdot 2 + 3 \cdot 1 + 3 + 1 + 1 + 3 \cdot 1 + 3 + 3 + 3 = 26 = |E|$, como debe ser, porque cada arista aporta exactamente una unidad a un grado de entrada.

En el paso 1 sale `bwa_index`: sus tres sucesores bajan de 2 a 1 y ninguno se libera. En el paso 2 sale `fastpA`: `mapearA` baja a 0 y entra en la cola, mientras que `multiqc` baja de 3 a 2. Tras los pasos 3 y 4 se han liberado los tres mapeos y `multiqc`. Así continúa (figura 18.2) hasta el paso 19, en que sale `all` y la cola queda vacía con los 19 trabajos ordenados. Si en lugar de una cola procesamos rondas completas (todas las fuentes de una vez), obtenemos 8 rondas de anchos 4, 4, 3, 1, 1, 4, 1, 1: las columnas de la figura 18.1. El ancho máximo, 4, es el número de núcleos a partir del cual este flujo ya no puede aprovechar más paralelismo, un resultado que precisaremos en la sección 18.1.10.

kahn.py

```
from collections import deque

def orden_topologico(sucesores):
    """Kahn. sucesores = {nodo: [hijos]}; error si hay un ciclo."""
    grado = {v: 0 for v in sucesores}
    for hijos in sucesores.values():
        for u in hijos:
            grado[u] = grado.get(u, 0) + 1
    cola = deque(v for v, g in grado.items() if g == 0)
    orden = []
    while cola:
        v = cola.popleft()
```

paso	sale de Q	grado llega a 0	cola Q tras el paso
0			bwa_index fastp_A fastp_B fastp_C
1	bwa_index	—	fastp_A fastp_B fastp_C
2	fastp_A	mapear_A	fastp_B fastp_C mapear_A
3	fastp_B	mapear_B	fastp_C mapear_A mapear_B
4	fastp_C	mapear_C multiqc	mapear_A mapear_B mapear_C multiqc
5	mapear_A	dedup_A	mapear_B mapear_C multiqc dedup_A
6	mapear_B	dedup_B	mapear_C multiqc dedup_A dedup_B
7	mapear_C	dedup_C	multiqc dedup_A dedup_B dedup_C
8	multiqc	—	dedup_A dedup_B dedup_C
9	dedup_A	—	dedup_B dedup_C
10	dedup_B	—	dedup_C
11	dedup_C	llamar	llamar
12	llamar	filtrar	filtrar
13	filtrar	anotar consenso_A consenso_B consenso_C	anotar consenso_A consenso_B consenso_C
14	anotar	—	consenso_A consenso_B consenso_C
15	consenso_A	—	consenso_B consenso_C
16	consenso_B	—	consenso_C
17	consenso_C	filogenia	filogenia
18	filogenia	all	all
19	all	—	vacía

Figura 18.2. El algoritmo de Kahn aplicado al grafo de la figura 18.1, con una cola FIFO y desempate por el orden en que aparecen las reglas en el Snakefile. En el paso 0 la cola contiene las cuatro fuentes (grado de entrada 0). Cada fila extrae un trabajo, decrementa el grado de sus sucesores y encola los que llegan a cero. Observe tres detalles: mapear_A solo se libera en el paso 2, cuando ya salieron sus *dos* predecesores (el índice y fastp_A); multiqc espera a los tres fastp (paso 4); y llamar espera a los tres dedup (paso 11). La última columna es, literalmente, la lista de trabajos que un planificador podría lanzar en ese instante. Tabla generada con `figuras/cap18/generar.py`.

```

orden.append(v)
for u in sucesores.get(v, []):
    grado[u] -= 1
    if grado[u] == 0:
        cola.append(u)
if len(orden) < len(grado):                                # teorema, punto (iii)
    resto = [v for v, g in grado.items() if g > 0]
    raise ValueError(f"cíclo entre: {resto}")
return orden

```

18.1.4 Dependencias por archivos: la herencia de Make

La idea de describir un cálculo como un grafo de archivos es mucho más antigua que la bioinformática. **make**, escrito en los Laboratorios Bell en la segunda mitad de los setenta y descrito por Feldman (1979), nació para recompilar programas en C: un *makefile* declara, para cada *objetivo*, de qué *prerrequisitos* depende y qué orden lo fabrica. **make** reconstruye el grafo, lo ordena y ejecuta solo lo necesario. El criterio de “necesario” es la clave de toda la familia de herramientas que le siguió:

Definición 18.2. (Objetivo desactualizado)

Sea $\text{pre}(t)$ el conjunto de prerrequisitos del objetivo t y $\tau(f)$ la fecha de última modificación del archivo f . El objetivo t debe reconstruirse si y solo si

$$\text{rehacer}(t) = \neg \text{existe}(t) \vee \bigvee_{d \in \text{pre}(t)} [\text{rehacer}(d) \vee \tau(d) > \tau(t)]. \quad (18.3)$$

DONDE

$\text{pre}(t)$	Archivos de los que depende t (aristas entrantes en el grafo de archivos).
$\tau(f)$	Marca de tiempo de modificación (<i>mtime</i>) que guarda el sistema de archivos.
$\neg, \vee$	Negación y disyunción lógicas.

La definición es recursiva y se evalúa precisamente en orden topológico: cuando llegamos a t , ya sabemos si cada uno de sus prerrequisitos se rehace. Su consecuencia práctica es que un cambio se propaga a *todos los descendientes* del archivo modificado en el grafo, y a nadie más. En la figura 18.1, modificar las lecturas de la muestra A obliga a rehacer 11 de los 18 trabajos reales (su **fastp**, su mapeo y su **dedup**, el

llamado conjunto y todo lo que cuelga de él, y `multiqc`); cambiar el umbral de `filtrar`, solo 6; retocar `multiqc`, solo uno. Esta es la *mise en place* de la intuición inicial.

🔗 De los compiladores al genoma

La idea de `make` era sencilla: describir una vez las dependencias entre archivos y dejar que el programa, comparando fechas de modificación, decida qué recompilar (Feldman, 1979). Cuatro décadas después, el problema que resuelve sigue vivo con otros nombres: una tabla de conteos calculada con una versión vieja del GTF, una figura que no se regeneró tras corregir un filtro. Muchos bioinformáticos usaron `make` para sus flujos durante años, pero sus limitaciones (un único comodín % por regla, un lenguaje propio poco expresivo, ninguna noción de clúster ni de entornos de software) motivaron la generación de gestores que estudiamos a continuación.

18.1.5 Snakemake: reglas, comodines e inferencia hacia atrás

Snakemake (Köster & Rahmann, 2012) conserva la filosofía de `make` (reglas que producen archivos a partir de archivos) pero la escribe en un lenguaje que es una extensión de Python. Cada rule declara sus input, output y la orden que los conecta (shell, run con Python o script con un archivo externo), y puede añadir threads, log, params, resources o el entorno de software (conda, container). La revisión de Mölder et al. (2021) describe cómo el diseño persigue un análisis *sostenible*, que combine reproducibilidad, adaptabilidad y transparencia.

Comodines. La pieza que hace escalar una regla es el *comodín* (*wildcard*): un nombre entre llaves dentro de un patrón de archivo, como `mapeo/{m}.dedup.bam`. Formalmente, un patrón p con comodines $w_1, \dots, w_k$ define una expresión regular $\rho(p)$ en la que cada $\{w_i\}$ se sustituye por un grupo con nombre (por defecto, $.$) y el resto del texto se toma literalmente. Un archivo f coincide con p si $\rho(p)$ reconoce f completo, y la coincidencia asigna valores $\sigma = \{w_i \mapsto s_i\}$ que luego se sustituyen en los patrones de entrada:

$$f \in \mathcal{L}(\rho(p_{\text{sal}})) \implies \text{entradas}(f) = \{\sigma(p) : p \in P_{\text{ent}}\}. \quad (18.4)$$

DONDE

$p_{\text{sal}}, P_{\text{ent}}$	Patrón de salida de la regla y conjunto de sus patrones de entrada.
$\rho(p)$	Expresión regular asociada al patrón p .
$\mathcal{L}(\rho)$	Lenguaje reconocido por ρ : el conjunto de nombres de archivo que coinciden.
σ	Asignación de valores a los comodines obtenida de la coincidencia.

Inferencia hacia atrás. A diferencia de un *script*, Snakemake no empieza por el principio sino por el final. Parte de los archivos objetivo (las entradas de la primera regla, por convención llamada `all`) y, para cada uno, busca la regla cuya salida coincide; con la asignación σ calcula las entradas, y repite el proceso recursivamente con cada entrada hasta llegar a archivos que ya existen. El grafo de trabajos de la figura 18.1 *no está escrito en ninguna parte*: se deduce de los nombres de archivo. Es un modelo *bajo demanda* (*pull*): solo se construye lo que alguien pidió. El siguiente programa reproduce la idea en unas pocas líneas.

```
inferir_dag.py

import re

REGLAS = [ # (nombre, salidas, entradas)
    ("fastp", ["limpio/{m}_R1.fq.gz", "limpio/{m}_R2.fq.gz"],
             ["datos/{m}_R1.fastq.gz", "datos/{m}_R2.fastq.gz"]),
    ("mapear", ["mapeo/{m}.ordenado.bam"],
             ["limpio/{m}_R1.fq.gz", "limpio/{m}_R2.fq.gz"]),
    ("dedup", ["mapeo/{m}.dedup.bam"], ["mapeo/{m}.ordenado.bam"]),
]

def a_regex(patron):
    # 'mapeo/{m}.bam' -> 'mapeo/(?P<m>.+)\.bam$'
    partes = re.split(r"\{(\w+)\}", patron)
    rx = "".join(re.escape(p) if i % 2 == 0 else f"(?P<{p}>.+)"
                 for i, p in enumerate(partes))
    return re.compile(rx + "$")

def inferir(objetivo, dag):
    for nombre, salidas, entradas in REGLAS:
        for s in salidas:
            hallado = a_regex(s).match(objetivo)
            if hallado:
                comodines = hallado.groupdict()
```

```

        ins = [e.format(**comodines) for e in entradas]
        dag[objetivo] = (nombre, comodines, ins)
        for f in ins:
            if f not in dag:
                inferir(f, dag)
        return
    dag[objetivo] = ("(archivo fuente)", {}, [])

dag = {}
inferir("mapeo/A.dedup.bam", dag)
for f, (regla, w, ins) in dag.items():
    print(f"{f:24s} <- {regla:16s} {w}")

```

Salida de inferir_dag.py

```

mapeo/A.dedup.bam      <- dedup      {'m': 'A'}
mapeo/A.ordenado.bam   <- mapear     {'m': 'A'}
limpio/A_R1.fq.gz      <- fastp       {'m': 'A'}
datos/A_R1.fastq.gz    <- (archivo fuente) {}
datos/A_R2.fastq.gz    <- (archivo fuente) {}
limpio/A_R2.fq.gz      <- fastp       {'m': 'A'}

```

Nuestra versión de juguete no distingue que los dos archivos de lecturas limpias de la muestra A salen de *un mismo* trabajo; Snakemake sí, porque agrupa todas las salidas de una regla con la misma asignación σ . Tampoco resuelve ambigüedades: si dos reglas pueden producir el mismo archivo, Snakemake se detiene con un error y pide al autor que las ordene (`ruleorder`) o que restrinja los comodines con expresiones regulares (`wildcard_constraints`).

El flujo completo. El *Snakefile* siguiente implementa el grafo de la figura 18.1. Cada regla es un paso que ya conocemos: `fastp` (??), BWA-MEM y `samtools` (??), `bcftools` y `Snpeff` (??) e IQ-TREE (??). Las muestras, la referencia y los umbrales se leen de un archivo de configuración, de modo que el mismo código sirve para 3 o para 300 aislados. Hemos evitado a propósito las tuberías con “|” dentro de las órdenes, para que cada archivo intermedio sea visible; en producción se sustituirían por tuberías o por la función `pipe()` de Snakemake.

Snakefile

```
# De lecturas crudas a variantes anotadas y filogenia
```

```

configfile: "config/config.yaml"

MUESTRAS = config["muestras"]          # p. ej. ["A", "B", "C"]
REF = config["referencia"]              # p. ej. "ref/genoma.fa"

wildcard_constraints:
    m="[A-Za-z0-9-]+"

rule all:
    input:
        "resultados/anotado.vcf",
        "resultados/arbol.treefile",
        "resultados/multiqc.html",

rule fastp:
    input:
        r1="datos/{m}_R1.fastq.gz",
        r2="datos/{m}_R2.fastq.gz",
    output:
        r1=temp("limpio/{m}_R1.fq.gz"),
        r2=temp("limpio/{m}_R2.fq.gz"),
        js="qc/{m}.fastp.json",
    log: "logs/fastp/{m}.log"
    threads: 4
    conda: "envs/qc.yaml"
    shell:
        "fastp -i {input.r1} -I {input.r2} -w {threads} "
        "-o {output.r1} -O {output.r2} -j {output.js} 2> {log}"

rule bwa_index:
    input: REF
    output: multiext(REF, ".amb", ".ann", ".bwt", ".pac", ".sa")
    conda: "envs/mapeo.yaml"
    shell: "bwa index {input}"

rule mapear:
    input:
        ref=REF,
        idx=rules.bwa_index.output,
        r1="limpio/{m}_R1.fq.gz",
        r2="limpio/{m}_R2.fq.gz",

```

```

output: temp("mapeo/{m}.ordenado.bam")
params: rg=lambda wc: f"@RG\\tID:{wc.m}\\tSM:{wc.m}"
log: "logs/mapear/{m}.log"
threads: 8
conda: "envs/mapeo.yaml"
shell:
    """
    bwa mem -t {threads} -R '{params.rg}' {input.ref} \
        {input.r1} {input.r2} > {output}.sam 2> {log}
    samtools fixmate -m {output}.sam {output}.fm.bam
    samtools sort -@ {threads} -o {output} {output}.fm.bam
    rm {output}.sam {output}.fm.bam
    """

```

rule dedup:

```

input: "mapeo/{m}.ordenado.bam"
output:
    bam="mapeo/{m}.dedup.bam",
    bai="mapeo/{m}.dedup.bam.bai",
conda: "envs/mapeo.yaml"
shell:
    "samtools markdup {input} {output.bam} && "
    "samtools index {output.bam}"

```

rule llamar:

```

input:
    ref=REF,
    bams=expand("mapeo/{m}.dedup.bam", m=MUESTRAS),
    bais=expand("mapeo/{m}.dedup.bam.bai", m=MUESTRAS),
output: "variantes/crudo.bcf"
conda: "envs/variantes.yaml"
shell:
    """
    bcftools mpileup -f {input.ref} -a AD,DP -Ob \
        -o {output}.pila {input.bams}
    bcftools call --ploidy 1 -mv -V indels -Ob \
        -o {output} {output}.pila
    rm {output}.pila
    """

```

rule filtrar:

```

input: "variantes/crudo.bcf"

```

```

output:
    vcf="variantes/filtrado.vcf.gz",
    tbi="variantes/filtrado.vcf.gz.tbi",
params: expr=config["filtro"]      # "QUAL>=30 && INFO/DP>=10"
conda: "envs/variantes.yaml"
shell:
    "bcftools filter -i '{params.expr}' -Oz -o {output.vcf} "
    "{input} && bcftools index -t {output.vcf}"

```

```

rule anotar:
    input: "variantes/filtrado.vcf.gz"
    output: "resultados/anotado.vcf"
    params: db=config["snpeff_db"]
    conda: "envs/anotacion.yaml"
    shell: "snpeff ann -noStats {params.db} {input} > {output}"

```

```

rule consenso:
    input:
        ref=REF,
        vcf="variantes/filtrado.vcf.gz",
        tbi="variantes/filtrado.vcf.gz.tbi",
    output: "consenso/{m}.fa"
    conda: "envs/variantes.yaml"
    shell:
        """
        bcftools consensus -f {input.ref} -s {wildcards.m} \
        -o {output}.tmp {input.vcf}
        sed 's/^>.*>{wildcards.m}/' {output}.tmp > {output}
        rm {output}.tmp
        """

```

```

rule filogenia:
    input:
        ref=REF,
        fas=expand("consenso/{m}.fa", m=MUESTRAS),
    output:
        arbol="resultados/arbol.treefile",
        aln="resultados/aln.fa",
    threads: 4
    conda: "envs/filogenia.yaml"
    shell:
        """

```

```
cat {input.ref} {input.fas} > {output.aln}
iqtree2 -s {output.aln} -m GTR+G -B 1000 \
  -T {threads} --prefix resultados/arbol
"""
```

```
rule multiqc:
    input: expand("qc/{m}.fastp.json", m=MUESTRAS)
    output: "resultados/multiqc.html"
    conda: "envs/qc.yaml"
    shell: "multiqc qc -n multiqc.html -o resultados"
```

Vale la pena detenerse en algunos detalles. `expand()` genera la lista de archivos de todas las muestras y es lo que crea las aristas “muchos a uno” hacia `llamar`, `filogenia` y `multiqc`. `temp()` marca archivos que se borran en cuanto ningún trabajo pendiente los necesita, lo que ahorra disco sin romper la reanudación. `--ploidy 1` y `-V indels` reflejan que trabajamos con un genoma bacteriano haploide y que, para la filogenia, solo usamos SNV: así las secuencias consenso tienen la misma longitud que la referencia y, concatenadas, forman directamente un alineamiento. Y el orden de las reglas en el archivo es irrelevante (salvo la primera, que define el objetivo por defecto): el orden de ejecución lo decide el grafo.

Explorar y ejecutar el flujo con Snakemake

```
snakemake -n                # ensayo en seco: qué trabajos y por qué
snakemake --dag > dag.dot    # grafo de trabajos en formato Graphviz
dot -Tpdf dag.dot -o dag.pdf
snakemake --cores 16 --sdm conda # entornos conda por regla
snakemake --cores 16 -R filtrar # rehace filtrar y descendientes
snakemake --report informe.html # informe HTML con procedencia
```

⚠ Versiones de Snakemake y opciones de la línea de órdenes

La interfaz cambió en la versión 8: la antigua `--use-conda` se escribe ahora `--software-deployment-method conda` (abreviada `--sdm conda`), y el envío a clústeres pasó a *complementos* de ejecución (`--executor slurm`, por ejemplo). Consulte siempre `snakemake --help` de la versión que tenga fijada en su entorno y anote esa versión en el README.

18.1.6 Nextflow: procesos, canales y flujo de datos

Nextflow (Di Tommaso et al., 2017) parte de otro modelo de computación: el *flujo de datos* (*dataflow*). En lugar de reglas que producen archivos con nombre, hay *procesos* aislados que se comunican a través de *canales*, colas asíncronas por las que circulan valores (rutas de archivos, cadenas, tuplas). Un proceso se *dispara* cada vez que encuentra un elemento disponible en cada uno de sus canales de entrada, y deposita sus resultados en canales de salida que alimentan a otros procesos. No hay que declarar nombres de archivos intermedios ni reconstruir el grafo hacia atrás: el grafo se desenvuelve hacia adelante, a medida que los datos llegan. Es un modelo *por empuje* (*push*).

Definición 18.3. (Canales de cola y de valor)

Un *canal de cola* es una secuencia finita de elementos que se consumen: cada elemento es leído por un único disparo del proceso. Un *canal de valor* contiene un solo elemento que puede leerse un número ilimitado de veces. Si un proceso tiene canales de entrada de cola $c_1, \dots, c_k$ (y quizá otros de valor), el número de tareas que ejecuta es

$$n_{\text{tareas}} = \min_{1 \leq i \leq k} |c_i|, \quad (18.5)$$

y vale 1 si todas sus entradas son canales de valor.

DONDE

c_i	i -ésimo canal de entrada <i>de cola</i> del proceso.
$ c_i $	Número de elementos que llegan por ese canal.
n_{tareas}	Número de veces que el proceso se ejecuta (tareas).

La ecuación (18.5) explica el error más frecuente de quien empieza con Nextflow. Si el índice de BWA saliera por un canal de *cola* (con un único elemento) y las lecturas por otro canal de cola con tres elementos, el proceso de mapeo se ejecutaría $\min(1, 3) = 1$ vez: solo se mapearía la primera muestra, sin ningún mensaje de error. La solución es que el índice viaje por un canal de *valor*, que se “relee” para cada muestra. En Nextflow, un proceso cuyas entradas son todas canales de valor produce canales de valor, y el operador `collect()` convierte una cola en un único valor (la lista completa), que es justo lo que necesita el llamado conjunto. La figura 18.3 dibuja este flujo de datos.

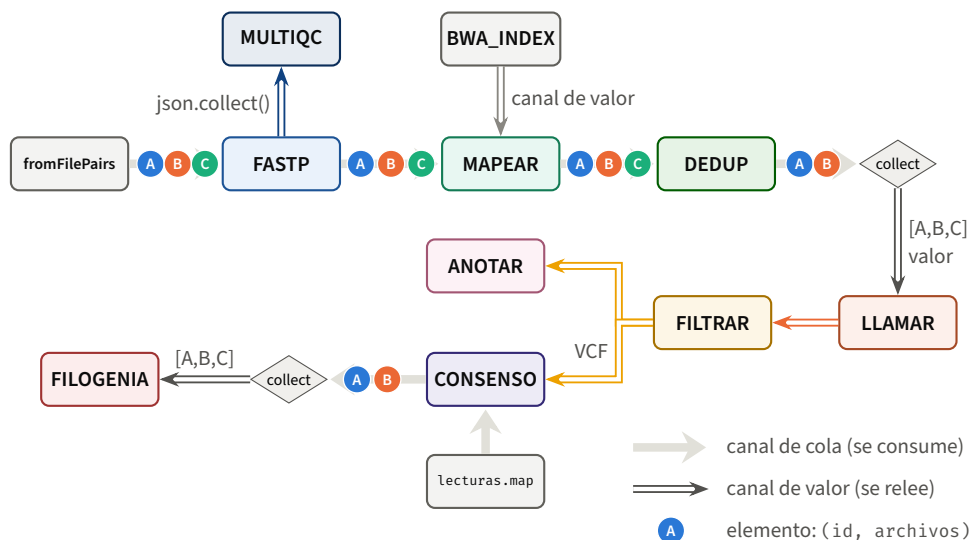


Figura 18.3. El mismo flujo de la figura 18.1 visto como flujo de datos en Nextflow. Los procesos (cajas) están aislados y solo se comunican por canales. Por los canales de cola (tubos grises) circulan elementos, uno por muestra, que cada proceso consume en cuanto llegan: la muestra A puede estar ya en DEDUP mientras C sigue en FASTP. El índice de BWA viaja por un canal de valor (doble línea) que MAPEAR relee para cada muestra, y el operador `collect` convierte la cola de BAM en un único valor, la lista completa, que dispara una sola vez el llamado conjunto. Lo mismo ocurre con el VCF filtrado, que CONSENSO relee para cada identificador de muestra, y con las secuencias consenso, que se reúnen antes de la filogenia.

El flujo completo en Nextflow. El archivo `main.nf` siguiente es el equivalente del Snakefile anterior, escrito en la sintaxis DSL2 (el lenguaje de Nextflow es una extensión de Groovy). Cada `process` declara sus entradas, sus salidas (con nombres `emit` para distinguirlas) y un bloque `script`; el bloque `workflow` final conecta los procesos con canales, como si fueran funciones.

```
main.nf

params.lecturas = "datos/*_R{1,2}.fastq.gz"
params.ref      = "ref/genoma.fa"
params.snpeff   = "Mycobacterium_tuberculosis_h37rv"
params.outdir   = "resultados"

process FASTP {
```

```

tag "${id}"
conda "envs/qc.yaml"
cpus 4
input:
tuple val(id), path(reads)
output:
tuple val(id), path("${id}_R*.fq.gz"), emit: reads
path "${id}.fastp.json", emit: json
script:
"""
fastp -i ${reads[0]} -I ${reads[1]} -w ${task.cpus} \
-o ${id}_R1.fq.gz -O ${id}_R2.fq.gz -j ${id}.fastp.json
"""
}

process BWA_INDEX {
conda "envs/mapeo.yaml"
input:
path fa
output:
tuple path(fa), path("${fa}.*")
script:
"""
bwa index ${fa}
"""
}

process MAPEAR {
tag "${id}"
conda "envs/mapeo.yaml"
cpus 8
input:
tuple val(id), path(reads)
tuple path(fa), path(idx)
output:
tuple val(id), path("${id}.ordenado.bam")
script:
"""
bwa mem -t ${task.cpus} -R '@RG\tID:${id}\tSM:${id}' \
${fa} ${reads} > ${id}.sam
samtools fixmate -m ${id}.sam ${id}.fm.bam
samtools sort -@ ${task.cpus} -o ${id}.ordenado.bam ${id}.fm.bam
"""
}

```

```

process DEDUP {
  tag "${id}"
  conda "envs/mapeo.yaml"
  input:
    tuple val(id), path(bam)
  output:
    tuple val(id), path("${id}.dedup.bam"), path("${id}.dedup.bam.bai")
  script:
    """
    samtools markdup ${bam} ${id}.dedup.bam
    samtools index ${id}.dedup.bam
    """
}

process LLAMAR {
  conda "envs/variantes.yaml"
  input:
    path bams
    path fa
  output:
    path "crudo.bcf"
  script:
    """
    bcftools mpileup -f ${fa} -a AD,DP -Ob -o pila.bcf *.dedup.bam
    bcftools call --ploidy 1 -mv -V indels -Ob -o crudo.bcf pila.bcf
    """
}

process FILTRAR {
  publishDir params.outdir, mode: 'copy'
  conda "envs/variantes.yaml"
  input:
    path bcf
  output:
    tuple path("filtrado.vcf.gz"), path("filtrado.vcf.gz.tbi")
  script:
    """
    bcftools filter -i 'QUAL>=30 && INFO/DP>=10' \\\
      -Oz -o filtrado.vcf.gz ${bcf}
    bcftools index -t filtrado.vcf.gz
    """
}

process ANOTAR {
  publishDir params.outdir, mode: 'copy'

```

```

conda "envs/anotacion.yaml"
input:
tuple path(vcf), path(tbi)
output:
path "anotado.vcf"
script:
"""
snpEff ann -noStats ${params.snpeff} ${vcf} > anotado.vcf
"""
}

process CONSENSO {
tag "${id}"
conda "envs/variantes.yaml"
input:
val id
tuple path(vcf), path(tbi)
path fa
output:
path "${id}.fa"
script:
"""
bcftools consensus -f ${fa} -s ${id} -o tmp.fa ${vcf}
sed 's/^>.*>${id}/' tmp.fa > ${id}.fa
"""
}

process FILOGENIA {
publishDir params.outdir, mode: 'copy'
conda "envs/filogenia.yaml"
cpus 4
input:
path fastas
path fa
output:
path "arbol.treefile"
script:
"""
cat ${fa} ${fastas} > aln.fa
iqtree2 -s aln.fa -m GTR+G -B 1000 -T ${task.cpus} --prefix arbol
"""
}

process MULTIQC {
publishDir params.outdir, mode: 'copy'

```

```

conda "envs/qc.yaml"
input:
path jsons
output:
path "multiqc.html"
script:
"""
multiqc . -n multiqc.html
"""
}

workflow {
  lecturas = Channel.fromFilePairs(params.lecturas) // cola
  ref = file(params.ref) // valor
  FASTP(lecturas)
  MULTIQC(FASTP.out.json.collect())
  BWA_INDEX(ref) // valor
  MAPEAR(FASTP.out.reads, BWA_INDEX.out)
  DEDUP(MAPEAR.out)
  bams = DEDUP.out.map { id, bam, bai -> [bam, bai] }.collect()
  LLAMAR(bams, ref)
  FILTRAR(LLAMAR.out)
  ANOTAR(FILTRAR.out)
  ids = lecturas.map { id, reads -> id }
  CONSENSO(ids, FILTRAR.out, ref)
  FILOGENIA(CONSENSO.out.collect(), ref)
}

```

Compare los dos archivos. En Snakemake, las dependencias *emergen* de los nombres de archivo; en Nextflow se *declaran* al conectar canales en el bloque workflow. Snakemake necesita conocer el grafo completo antes de empezar (lo construye hacia atrás desde el objetivo), mientras que Nextflow puede empezar a mapear la muestra A antes de saber cuántas muestras hay, lo que resulta natural cuando las muestras llegan por un flujo continuo. Cada tarea de Nextflow se ejecuta en su propio directorio de trabajo aislado, con enlaces a sus entradas, de modo que dos tareas nunca pisan los archivos de otra; publishDir copia a la carpeta de resultados solo lo que nos interesa conservar.

Ejecutar main.nf y un flujo de nf-core

```

nextflow run main.nf -with-conda -resume
nextflow run main.nf -profile docker -with-report -with-dag dag.html

```

```
# Flujo comunitario: fije siempre la version con -r
nextflow run nf-core/viralrecon -r <version> -profile test,docker \
--outdir resultados_prueba
```

nf-core y CWL. Sobre Nextflow creció nf-core (Ewels et al., 2020), una comunidad que mantiene flujos curados (RNA-seq, llamado de variantes, metagenómica, epigenómica...) con normas estrictas: cada flujo tiene un perfil `test` con datos diminutos que se ejecuta en integración continua, contenedores para cada herramienta, parámetros documentados con un esquema y versiones publicadas. Antes de escribir un flujo propio para un análisis estándar, vale la pena buscar si ya existe uno allí. En otra dirección, el *Common Workflow Language* (CWL) no es un gestor sino un *estándar* abierto para describir herramientas y flujos en YAML de forma declarativa, que varios motores distintos pueden ejecutar (Crusoe et al., 2022). Su lema es la portabilidad: el mismo documento debería producir el mismo resultado en un portátil, en un clúster o en la nube, y en cualquier motor que cumpla la especificación. La tabla 18.1 resume las diferencias.

18.1.7 Congelar el entorno: Bioconda y contenedores

Un flujo que funciona en mi portátil y no en el suyo no es reproducible. La causa casi nunca es el código del flujo, sino el *entorno*: la versión de `samtools`, de la biblioteca `htslib` contra la que se compiló, de Python, de la biblioteca matemática del sistema. Di Tommaso et al. (2017) mostraron que un mismo análisis ejecutado en sistemas operativos distintos podía producir resultados numéricamente diferentes, y que empaquetar las herramientas en contenedores eliminaba esa discrepancia. Grüning et al. (2018) proponen pensar la reproducibilidad como una pila de capas, de los paquetes al sistema operativo, y fijar cada una con la herramienta adecuada.

Gestores de paquetes. Conda instala binarios precompilados con todas sus dependencias en un entorno aislado, sin permisos de administrador. Bioconda (The Bioconda Team et al., 2018) es el canal comunitario de recetas para bioinformática: ya reúne miles de paquetes, y es la forma estándar de instalar casi cualquier herramienta de este libro. Cada regla del Snakefile o proceso de `main.nf` apunta a un pequeño archivo de entorno con versiones *exactas*:

Tabla 18.1. Comparación esquemática de cuatro formas de describir un flujo. Las cuatro construyen y ordenan un grafo acíclico dirigido; difieren en cómo se escribe, cómo se infiere y qué ecosistema las rodea. Para una comparación más amplia, véase Wratten et al. (2021).

	Make	Snakemake	Nextflow	CWL
Lenguaje	propio	Python + reglas	Groovy (DSL2)	YAML declarativo
Unidad	archivo	archivo con comodines	canal de valores	paso con entradas tipadas
Grafo	hacia atrás (<i>pull</i>)	hacia atrás (<i>pull</i>)	hacia adelante (<i>push</i>)	declarado explícitamente
Reanudación	marcas de tiempo	marcas de tiempo, parámetros, código	<i>hash</i> de cada tarea (-resume)	depende del motor
Entornos	ninguno	conda, contenedores	conda, contenedores	contenedores
Ejecución	local	local, clúster, nube	local, clúster, nube	según el motor
Comunidad	general	catálogo de flujos	nf-core	estándar multi-motor

```
envs/mapeo.yaml

channels:
  - conda-forge
  - bioconda
dependencies:
  - bwa=0.7.18
  - samtools=1.20
```

Un archivo así fija la versión de las herramientas que nombramos, pero no la de sus dependencias transitivas, que el resolutor elige en el momento de instalar. Para congelarlo todo se exporta un archivo de bloqueo con la lista explícita de paquetes (`conda list --explicit` o herramientas como `conda-lock`), o se da el paso siguiente: el contenedor.

Contenedores. Un contenedor empaqueta un sistema de archivos completo (binarios, bibliotecas, configuración) en una *imagen* que se ejecuta aislada sobre el núcleo del sistema anfitrión, sin el coste de una máquina virtual. Docker popularizó el formato, pero su demonio con privilegios de administrador lo hace inaceptable en la mayoría de los clústeres compartidos. Singularity (hoy también distribuido como Apptainer) se diseñó para ese entorno: el usuario dentro del contenedor es el mismo que fuera, no hace falta ningún demonio privilegiado y la imagen es un único archivo que se copia como cualquier otro (Kurtzer et al., 2017). El proyecto BioContainers (da Veiga Leprevost et al., 2017) construye automáticamente una imagen para cada paquete de Bioconda, de modo que la misma versión de una herramienta está disponible como paquete conda, como imagen Docker y como imagen Singularity.

⚠ La etiqueta `latest` no es una versión

Una imagen como `samtools:latest` cambia de contenido cada vez que alguien publica una versión nueva. Fije siempre una etiqueta de versión y, para máxima garantía, el *digest* de la imagen (`imagen@sha256:...`), que es la suma de verificación de su contenido y no puede cambiar. Lo mismo vale para los flujos de `nf-core` (`-r` con una versión) y para los entornos `conda` (versiones exactas, nunca solo el nombre del paquete).

18.1.8 Caché y reanudación

La ecuación (18.3) decide qué rehacer comparando marcas de tiempo. Es un criterio barato pero frágil: copiar un directorio, descomprimir un respaldo o tocar un archivo sin cambiar su contenido altera las fechas y desencadena recálculos innecesarios; y, al revés, cambiar un parámetro o la versión de una herramienta *no* altera ninguna fecha. Por eso las versiones modernas de Snakemake también vuelven a ejecutar un trabajo cuando cambian sus parámetros, su código o su entorno de software, y Nextflow abandona las fechas y usa un *hash* por tarea.

Definición 18.4. (Clave de caché de una tarea)

Sea H una función *hash* criptográfica. La clave de la tarea v es

$$k(v) = H(\text{orden}(v) \parallel \text{parámetros}(v) \parallel \text{entorno}(v) \parallel H(x_1) \parallel \dots \parallel H(x_r)), \quad (18.6)$$

donde $x_1, \dots, x_r$ son sus archivos de entrada. Si existe un resultado guardado con la clave $k(v)$, la tarea se da por hecha y se reutiliza.

DONDE

H	Función <i>hash</i> (p. ej., SHA-256): transforma cualquier cadena de bits en una huella de longitud fija.
$\parallel$	Concatenación de cadenas.
$\text{orden}(v)$	Texto de la orden o <i>script</i> de la tarea, ya con sus variables sustituidas.
$\text{entorno}(v)$	Identificador del contenedor o del entorno de software.
x_j	j -ésimo archivo de entrada de la tarea.

Como las entradas de una tarea son salidas de otras, un cambio en cualquier punto altera las claves de todos sus descendientes, y solo de ellos: la ecuación (18.6) reproduce la propagación de la ecuación (18.3) sin depender de relojes. En Nextflow, la opción `-resume` aplica esta idea con el directorio `work/`: por defecto, la huella de un archivo de entrada se calcula a partir de su ruta, tamaño y fecha (rápido), y con la directiva `cache 'deep'` a partir de su contenido (lento pero inmune a copias y restauraciones).

clave_tarea.py

```
import hashlib, json

def clave_tarea(orden, params, entorno, entradas):
    """Huella de una tarea al estilo de -resume (modo 'deep')."""
    h = hashlib.sha256()
    for parte in (orden, json.dumps(params, sort_keys=True), entorno):
        h.update(parte.encode())
    for ruta in sorted(entradas):
        with open(ruta, "rb") as fh:
            h.update(hashlib.sha256(fh.read()).digest())
    return h.hexdigest()[:12]
```

18.1.9 Ejecución en clúster y en la nube

El mismo grafo puede ejecutarse en un portátil, en un clúster con un gestor de colas (SLURM, PBS, SGE) o en la nube (AWS Batch, Google Cloud, Kubernetes). Ambos gestores separan la *lógica* del flujo de la *infraestructura*: en Snakemake mediante *perfiles* y complementos de ejecución; en Nextflow mediante el archivo `nextflow.config`, donde cada perfil elige el ejecutor, los contenedores y los recursos por proceso. El gestor se convierte en un planificador: mantiene la cola de trabajos listos del algoritmo de Kahn, envía cada uno al gestor de colas en cuanto se libera, vigila su terminación y reintenta los que fallan (por ejemplo, con más memoria).

```
nextflow.config

profiles {
  laptop {
    process.executor = 'local'
    conda.enabled = true
  }
  cluster {
    process.executor = 'slurm'
    process.queue = 'normal'
    singularity.enabled = true
    process {
      withName: 'MAPEAR' { cpus = 8; memory = '16 GB' }
      errorStrategy = 'retry'
      maxRetries = 2
    }
  }
}
```

18.1.10 Paralelismo: trabajo, camino crítico y la ley de Amdahl

¿Cuánto más rápido termina el flujo si pasamos de 4 a 64 núcleos? La pregunta tiene una respuesta precisa en términos del grafo. Asignemos a cada trabajo v una duración t_v y supongamos que cada trabajo ocupa un núcleo.

Definición 18.5. (Trabajo y camino crítico)

El *trabajo* de un flujo es la suma de las duraciones, $W = \sum_{v \in V} t_v$, que es el tiempo en un solo núcleo, $T_1 = W$. El *camino crítico* (*span*) es la duración del camino dirigido más largo,

$$S = \max_{\text{caminos } v_1 \rightarrow \dots \rightarrow v_\ell} \sum_{j=1}^{\ell} t_{v_j}, \quad (18.7)$$

que se calcula en tiempo $O(|V| + |E|)$ recorriendo el grafo en orden topológico con $F(v) = t_v + \max_{u \rightarrow v} F(u)$.

DONDE

t_v	Duración del trabajo v en un núcleo.
W, T_1	Trabajo total; tiempo de ejecución en serie.
S	Camino crítico: tiempo mínimo aun con infinitos núcleos.
$F(v)$	Instante más temprano en que puede terminar v .

Teorema 18.3. (Cotas del planificador voraz)

Sea T_P el tiempo que tarda un planificador voraz (que nunca deja un núcleo ocioso si hay un trabajo listo) con P núcleos. Entonces

$$\max\left(\frac{W}{P}, S\right) \leq T_P \leq \frac{W}{P} + S. \quad (18.8)$$

Demostración. La cota inferior es evidente: P núcleos no pueden hacer W unidades de trabajo en menos de W/P , y ningún planificador puede acortar la cadena de dependencias más larga. Para la superior, dividamos el tiempo en instantes *completos*, en que los P núcleos están ocupados, e *incompletos*. En los completos se realizan P unidades de trabajo por unidad de tiempo, así que suman como mucho W/P . En un instante incompleto, el planificador voraz está ejecutando *todos* los trabajos listos, entre ellos el primer trabajo pendiente de cualquier camino crítico del grafo restante; por lo tanto, la longitud de ese camino disminuye a la misma velocidad que el reloj, y los instantes incompletos suman como mucho S . $\square$

La ecuación (18.8) dice que la aceleración máxima alcanzable es W/S , el *paralelismo* del flujo, por muchos núcleos que compremos. Un caso particular famoso

se obtiene cuando una fracción s del trabajo es intrínsecamente serial y el resto es perfectamente divisible.

Teorema 18.4. (Ley de Amdahl)

Si una fracción s del tiempo en serie no puede paralelizarse y la fracción $1 - s$ se reparte sin coste entre P núcleos, la aceleración es

$$\psi(P) = \frac{T_1}{T_P} = \frac{T_1}{sT_1 + (1-s)T_1/P} = \frac{1}{s + \frac{1-s}{P}} \xrightarrow{P \rightarrow \infty} \frac{1}{s}. \quad (18.9)$$

DONDE

s	Fracción serial del trabajo ($0 < s \leq 1$).
P	Número de núcleos (o de trabajos simultáneos).
$\psi(P)$	Aceleración (<i>speedup</i>) con P núcleos respecto de uno.

El argumento es de Amdahl (1967), que lo usó para defender los procesadores rápidos frente a las máquinas masivamente paralelas: si el 10 % del trabajo es serial ($s = 0,1$), 8 núcleos dan una aceleración de 4,71, 64 núcleos apenas 8,77, y ninguna cantidad de núcleos supera 10. Gustafson (1988) replicó que en la práctica no se resuelve el mismo problema más deprisa, sino un problema *más grande* en el mismo tiempo: si con P núcleos la parte paralela crece en proporción a P y s es la fracción serial *medida en la máquina paralela*, la aceleración escalada es

$$\psi_G(P) = s + (1-s)P = P - s(P-1), \quad (18.10)$$

que con $s = 0,1$ y $P = 64$ vale 57,7. Ambas leyes son correctas; responden a preguntas distintas. En bioinformática se ven las dos: cuando añadimos muestras a un estudio crece sobre todo la parte paralela (Gustafson), pero cada paso conjunto que las reúne a todas (el llamado de variantes, la filogenia, la normalización de una matriz de conteos) añade un tramo serial que limita la aceleración de una ejecución concreta (Amdahl). Para diagnosticar un flujo real, Karp y Flatt (1990) propusieron estimar la fracción serial *efectiva* a partir de una aceleración medida:

$$e(P) = \frac{1/\psi(P) - 1/P}{1 - 1/P}. \quad (18.11)$$

Si e se mantiene constante al aumentar P , el límite es una parte serial genuina; si crece con P , el problema es la sobrecarga de coordinación.

Ejemplo 18.2. (¿Cuántos núcleos pedir para el proyecto?)

Asignemos al flujo de la figura 18.1 duraciones ilustrativas (en minutos) por trabajo: **fastp** 3, **mapear** 12, **dedup** 4 y **consenso** 1 por muestra; **bwa_index** 2, **filtrar** 1, **anotar** 2 y **multiqc** 1; y, como los pasos conjuntos crecen con el número n de muestras, **llamar** $6 + 3n$ y **filogenia** $4 + 2n$.

Tres muestras. El trabajo es $W = 91$ min y el camino crítico, **fastp** → **mapear** → **dedup** → **llamar** → **filtrar** → **consenso** → **filogenia**, suma $S = 3 + 12 + 4 + 15 + 1 + 1 + 10 = 46$ min. El paralelismo es $W/S = 1,98$: nunca terminaremos en menos de 46 minutos. Con $P = 2$ el planificador voraz tarda 61 min (dentro de las cotas $[45,5; 91,5]$) y con $P = 4$ alcanza ya los 46 min, el mínimo absoluto. Pedir 8 núcleos sería desperdiciar la mitad.

Veinticuatro muestras. Ahora $W = 616$ min y $S = 151$ min, así que $W/S = 4,08$. La simulación del planificador (figura 18.4) da aceleraciones de 3,19 con 8 núcleos, 3,71 con 16 y 4,08 a partir de 32. La fracción serial de Karp-Flatt con 16 núcleos es $e = 0,221$: más de una quinta parte del tiempo es serial. ¿Dónde? En **llamar** ($6 + 3 \cdot 24 = 78$ min) y **filogenia** (52 min), que suman 130 de los 151 min del camino crítico. Si dividimos el llamado en 24 regiones del genoma que se procesan en paralelo y luego se concatenan (*scatter-gather*, 1 min adicional), el camino crítico cae a 77,25 min, el paralelismo sube a 7,99 y la aceleración con 16 núcleos pasa de 3,71 a 6,46 ($e = 0,098$). El siguiente cuello de botella es la filogenia, que habría que paralelizar internamente (IQ-TREE admite varios hilos).

El tiempo de un flujo en paralelo está entre $\max(W/P, S)$ y $W/P + S$. Más núcleos solo ayudan mientras W/P domine; después, la única forma de ir más rápido es acortar el camino crítico.

Planificar no es trivial

Encontrar el orden que minimiza T_P para un grafo arbitrario con $P \geq 2$ es un problema NP-difícil, por lo que los gestores usan heurísticas. La más común, la que empleamos en la simulación de la figura 18.4, da prioridad al trabajo listo con el camino más largo

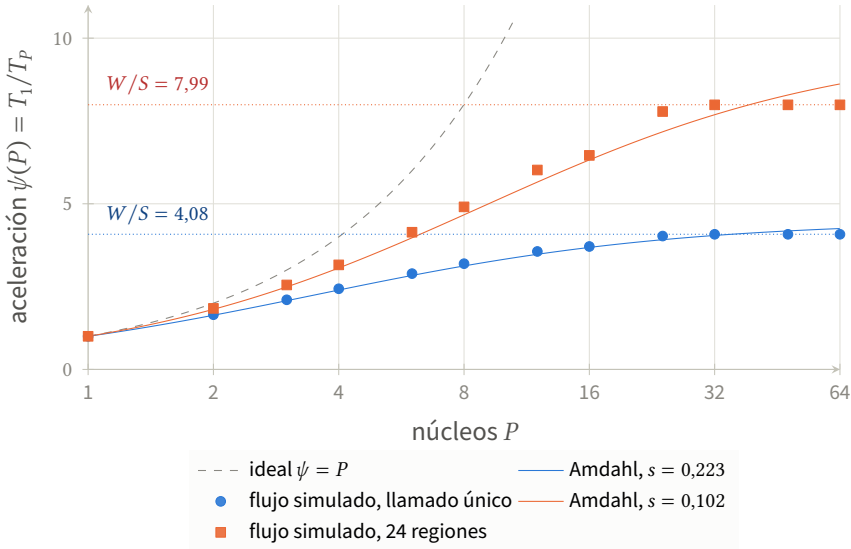


Figura 18.4. Aceleración del flujo de ejemplo con 24 muestras en función del número de núcleos, obtenida simulando un planificador voraz sobre el grafo de trabajos (puntos), junto a la curva de Amdahl (ecuación (18.9)) con la fracción serial que mejor se ajusta (líneas continuas). Con el llamado de variantes como un único trabajo (azul) la aceleración se estanca en el paralelismo $W/S = 4,08$ desde unos 32 núcleos. Dividir el llamado en 24 regiones (naranja) reduce la fracción serial a la mitad y casi duplica la aceleración alcanzable. La lección práctica: antes de pedir más núcleos, busque el camino crítico. Datos de `figuras/cap18/generar.py`.

hasta el final (*critical-path first*); la ecuación (18.8) garantiza que *cualquier* planificador voraz queda a menos de un factor 2 del óptimo, porque $W/P + S \leq 2 \max(W/P, S)$. En la práctica, la planificación real importa menos que otros costes que el modelo ignora: la espera en la cola del clúster, la lectura y escritura de archivos en un sistema de archivos compartido y el arranque de contenedores, que pueden dominar cuando los trabajos son muy cortos. De ahí la recomendación de agrupar trabajos diminutos (`group` en Snakemake, `array` o procesos por lotes en Nextflow).

18.1.11 Buenas prácticas y principios FAIR

Un gestor de flujos resuelve la parte mecánica de la reproducibilidad, pero no sustituye a los hábitos. Sandve et al. (2013) los condensaron en diez reglas que siguen vigentes: registrar cómo se produjo cada resultado; evitar los pasos manuales de manipulación de datos; archivar las versiones exactas de los programas externos;

poner bajo control de versiones todos los *scripts* propios; guardar los resultados intermedios, cuando sea posible, en formatos estándar; anotar las semillas de todo análisis con componentes aleatorios; guardar los datos crudos detrás de cada gráfica; generar salidas jerárquicas que permitan inspeccionar los detalles; conectar cada afirmación del texto con el resultado que la sustenta; y dar acceso público a *scripts*, ejecuciones y resultados. Wilson et al. (2017), pensando en quien no es informático de profesión, proponen un mínimo “suficientemente bueno”: conservar los datos crudos intactos, dar a cada archivo un nombre informativo, escribir código en funciones pequeñas con nombres claros, usar control de versiones y documentar el proyecto con un README que explique cómo reproducirlo.

Los datos y los resultados, por su parte, deben ser FAIR (Wilkinson et al., 2016): localizables (*Findable*: con identificadores persistentes y metadatos ricos, indexados en un buscador), accesibles (*Accessible*: recuperables por su identificador mediante un protocolo abierto, aunque el acceso esté controlado), interoperables (*Interoperable*: en formatos y vocabularios compartidos) y reutilizables (*Reusable*: con licencia clara, procedencia detallada y metadatos que sigan los estándares de la comunidad). Un aspecto que suele pasarse por alto es que los principios se dirigen tanto a las personas como a las *máquinas*: un conjunto de datos es FAIR cuando un programa puede encontrarlo, descargarlo y entenderlo sin intervención humana. Un flujo bien escrito es la contraparte computacional de esa idea: describe, de forma ejecutable por una máquina, cómo se pasa de los datos a los resultados.

18.2 Proyecto integrador

 Colab · 18.2 · Proyecto integrador

 repositorio del curso

Una receta que otro pueda cocinar

Un buen libro de cocina no dice “añada sal al gusto y hornee hasta que esté listo”. Dice cuántos gramos, a qué temperatura, durante cuántos minutos y en qué molde; y la prueba de que la receta está bien escrita es que alguien que nunca vio al autor la cocine y obtenga el mismo plato. El proyecto integrador es su primera receta de ese tipo. No se evalúa solo si su plato está rico (si sus resultados son biológicamente interesantes), sino si un compañero, con su repositorio y una sola orden, obtiene exactamente las mismas tablas y figuras.

Tabla 18.2. Las dos opciones del proyecto integrador, sus etapas y los capítulos del libro en los que se apoya cada una.

Etapas	A: patógeno (ADN)	B: RNA-seq
Datos	FASTQ pareados de aislados (SRA/ENA)	FASTQ de un diseño con réplicas
Calidad	<code>fastp</code> , MultiQC (??)	<code>fastp</code> , MultiQC (??)
Lecturas → señal	BWA-MEM, <code>samtools</code> (??)	pseudoalineamiento o mapeo, algoritmo EM (??)
Inferencia	llamado y filtrado de variantes (??)	normalización, binomial negativa, FDR (??)
Interpretación	anotación funcional, resistencia (??)	enriquecimiento GO/KEGG (??)
Síntesis	filogenia de consensos (??)	mapa de calor, rutas enriquecidas

18.2.1 Elegir la pregunta y dimensionar el proyecto

Todo proyecto empieza por una pregunta biológica concreta, y el resto (datos, métodos, recursos) se deriva de ella. Proponemos dos opciones que recorren buena parte del libro; la tabla 18.2 las compara.

- **Opción A: genómica de un patógeno.** A partir de lecturas Illumina pareadas de 20 a 30 aislados clínicos de una bacteria (por ejemplo, *Mycobacterium tuberculosis*, cuyo genoma de referencia H37Rv tiene unos 4,41 Mb), identificar variantes respecto a la referencia, anotar las que caen en genes asociados a resistencia antimicrobiana y reconstruir la filogenia de los aislados para discutir si hay transmisión reciente. Es exactamente el flujo de la figura 18.1. Plataformas de vigilancia genómica como Nextstrain se construyen sobre este mismo esquema, con Snakemake como motor (Hadfield et al., 2018).
- **Opción B: transcriptómica.** A partir de un experimento de RNA-seq público con al menos tres réplicas biológicas por condición, cuantificar transcritos, detectar genes diferencialmente expresados con control de la tasa de falsos descubrimientos e interpretar la lista con un análisis de enriquecimiento funcional (??).

Antes de escribir una sola regla conviene estimar cuánto disco y cuánto cómputo hará falta. Las fórmulas de cobertura del ?? bastan.

Ejemplo 18.3. (Presupuesto de la opción A)

Queremos 24 aislados de *M. tuberculosis* ($G = 4,41$ Mb) a una profundidad media de $c = 100\times$ con lecturas pareadas de 2×150 pb. El número de pares por muestra es $N = cG/(2L) = 100 \times 4,41 \times 10^6 / 300 \approx 1,47 \times 10^6$, es decir, $4,41 \times 10^8$ bases. En FASTQ, cada base ocupa aproximadamente dos bytes (la base y su calidad) más las cabeceras; con unas 60 letras de cabecera por lectura, el tamaño sin comprimir es de unos 1,06 GB por muestra, y comprimido con `gzip` (factor aproximado de 4) unos 0,27 GB. Los 24 aislados ocupan unos 6,4 GB de datos crudos: cabe en un portátil. Con las duraciones del ejemplo 18.2, el trabajo total es de $W = 616$ min $\approx 10,3$ horas-núcleo; con 16 núcleos, el planificador termina en unas 2,8 horas (1,6 horas si se divide el llamado por regiones). El proyecto es viable en un servidor modesto o en unas pocas horas de un clúster universitario. Con estas cifras en mano se puede decidir, antes de empezar, si los archivos intermedios deben borrarse (`temp()`) y cuántos núcleos pedir.

18.2.2 Estructura del proyecto

Un proyecto computacional bien organizado se entiende al abrir su carpeta principal. Noble (2009) propuso una organización que se ha vuelto casi un estándar: una carpeta para datos, otra para resultados, otra para código y otra para documentación, con el principio de que los datos crudos son de *solo lectura* y todo lo demás puede regenerarse desde ellos con una única orden. La figura 18.5 adapta esa idea a un flujo con Snakemake; con Nextflow basta cambiar `workflow/Snakefile` por `main.nf` y `nextflow.config`.

18.2.3 Las capas de la reproducibilidad

Conviene precisar el vocabulario. Llamaremos *reproducible* a un análisis que, con los mismos datos y el mismo código, produce los mismos resultados; *replicable*, a un hallazgo que se sostiene cuando se repite el experimento con datos nuevos. La reproducibilidad es el requisito mínimo: si ni siquiera el propio análisis puede repetirse, no hay forma de saber si un resultado sorprendente es biología o un error. Peng (2011) la describió como un espectro que va desde la publicación sin nada más hasta el estándar de oro de código y datos enlazados y ejecutables; cada escalón hace el trabajo más verificable.

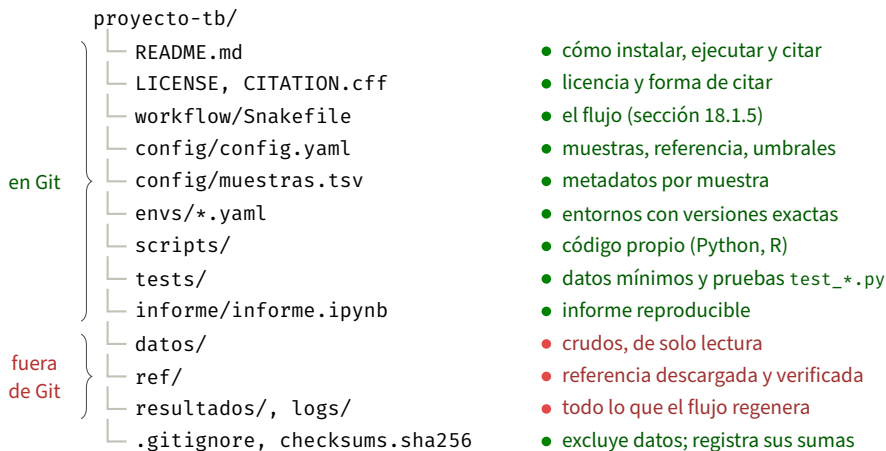


Figura 18.5. Estructura de directorios recomendada para el proyecto integrador, inspirada en Noble (2009). En verde, lo que se versiona con Git: código, configuración, entornos, pruebas y documentación, todo texto ligero. En rojo, lo que *no* se versiona: los datos crudos (que se descargan con un *script* a partir de sus accesiones y se verifican con sumas SHA-256), la referencia y todo lo que el flujo regenera. La regla de oro es que borrar `resultados/` nunca debe ser una catástrofe: una orden lo reconstruye.

Formalmente, el resultado R de un análisis es una función de todo lo que lo determina:

$$R = f\left(\underbrace{C}_{\text{código}}, \underbrace{\mathcal{E}}_{\text{entorno}}, \underbrace{D}_{\text{datos}}, \underbrace{\theta}_{\text{parámetros}}, \underbrace{\omega}_{\text{semilla}} \right). \quad (18.12)$$

DONDE

C	Código propio y del flujo; se fija con el identificador de un <i>commit</i> de Git.
$\mathcal{E}$	Entorno de software (paquetes, bibliotecas, sistema); se fija con entornos bloqueados o contenedores con <i>digest</i> .
D	Datos de entrada; se fijan con accesiones públicas y sumas de verificación.
θ	Parámetros y umbrales; se fijan en un archivo de configuración versionado.
ω	Estado inicial de los generadores de números aleatorios (??).

La ecuación (18.12) es trivial de escribir y exigente de cumplir: *cada* argumento que no se fije es una fuente de discrepancia. La figura 18.6 muestra las capas y

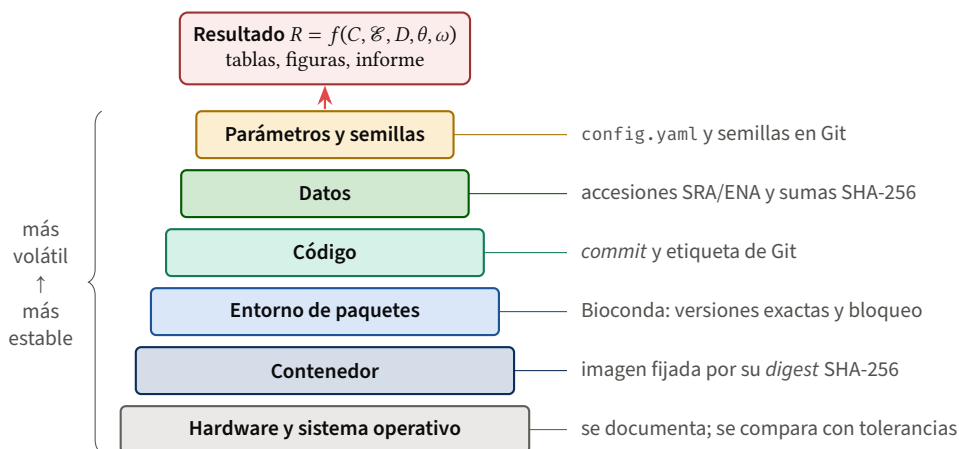


Figura 18.6. Las capas que determinan un resultado computacional (ecuación (18.12)) y la herramienta que fija cada una. Las capas inferiores cambian con menos frecuencia pero son las más difíciles de reconstruir años después; las superiores cambian en cada análisis y son las más fáciles de olvidar. Un contenedor fija de una vez las capas de entorno y parte del sistema, pero no los datos ni los parámetros; Git fija el código y la configuración, pero no el software. Solo la combinación de todas cierra la ecuación (18.12). Adaptado de la discusión de Grüning et al. (2018).

la herramienta que fija cada una. Hay, además, una fuente residual que ninguna herramienta elimina del todo: la aritmética de coma flotante no es asociativa. En Python, $(0.1 + 0.2) + 0.3$ da 0.6000000000000001 , mientras que $0.1 + (0.2 + 0.3)$ da 0.6 . Un programa que suma en paralelo en un orden que depende de qué hilo termina antes puede producir resultados que difieren en el último decimal entre ejecuciones; lo sensato es documentarlo y comparar resultados con tolerancias, no bit a bit.

Sumas de verificación. La capa de datos se fija con sumas de verificación: el flujo descarga las lecturas por su accesión y comprueba que su huella SHA-256 coincide con la registrada. ¿Qué tan seguras son? Si una función *hash* de b bits se comporta como una asignación aleatoria uniforme, la probabilidad de que entre n archivos distintos haya dos con la misma huella es, por el argumento del cumpleaños,

$$\mathbb{P}(\text{colisión}) = 1 - \prod_{i=1}^{n-1} \left(1 - \frac{i}{2^b}\right) \approx 1 - \exp\left(-\frac{n(n-1)}{2^{b+1}}\right). \quad (18.13)$$

DONDE

b	Longitud de la huella en bits (32 para CRC32, 128 para MD5, 256 para SHA-256).
n	Número de archivos distintos comparados.

La aproximación se obtiene tomando logaritmos, usando $\ln(1 - x) \approx -x$ para x pequeño y sumando $\sum_{i=1}^{n-1} i = n(n-1)/2$. Con una suma de 32 bits (como CRC32) y $n = 10^5$ archivos, la probabilidad de colisión ya es de 0,69: las sumas cortas sirven para detectar errores de transmisión, no para identificar contenidos. Con MD5 (128 bits) y mil millones de archivos, la probabilidad accidental es de $1,5 \times 10^{-21}$, y con SHA-256, de 4×10^{-60} . MD5 sigue siendo útil contra la corrupción accidental (el SRA y el ENA lo publican para cada archivo), pero hoy se pueden fabricar colisiones deliberadas, así que para garantías frente a manipulaciones se prefiere SHA-256.

Registrar y verificar sumas SHA-256

```
sha256sum datos/*.fastq.gz ref/genoma.fa > checksums.sha256
git add checksums.sha256           # la lista de sumas SI va en Git
sha256sum -c checksums.sha256      # en otra máquina: OK / FAILED
```

18.2.4 Control de versiones y pruebas

El código del proyecto vive en un repositorio Git desde el primer día (??). Las reglas de Perez-Riverol et al. (2016) resumen los hábitos que más rinden: *commits* pequeños con mensajes que expliquen el porqué, ramas para cada cambio sustancial, etiquetas (v1.0) para las versiones que producen resultados reportados y un repositorio público enlazado desde la publicación. Los datos y resultados no van en Git (figura 18.5), pero sí la lista de sus accesiones y sumas.

Las *pruebas* son la parte más descuidada de los análisis académicos, y la que más ahorra. Distinguimos tres niveles:

1. **Pruebas unitarias** de las funciones propias (en *scripts/*), con casos pequeños cuyo resultado se conoce de antemano, ejecutadas con *pytest*.
2. **Pruebas de integración** del flujo completo sobre un conjunto de datos *diminuto* (unos miles de lecturas simuladas sobre un fragmento de la referencia), que debe correr en segundos; es la idea de los perfiles test de nf-core (Ewels et al., 2020).

3. **Comprobaciones de cordura** sobre los resultados reales: la proporción de lecturas mapeadas, la razón T_i/T_v de las variantes (??), el número de genes detectados, la correlación entre réplicas.

```
tests/test_kahn.py

import pytest
from scripts.kahn import orden_topologico

def test_respeta_dependencias():
    g = {"fastp": ["mapear"], "indice": ["mapear"],
         "mapear": ["dedup"], "dedup": []}
    pos = {v: i for i, v in enumerate(orden_topologico(g))}
    for v, hijos in g.items():
        assert all(pos[v] < pos[u] for u in hijos)

def test_detecta_ciclos():
    with pytest.raises(ValueError):
        orden_topologico({"a": ["b"], "b": ["c"], "c": ["a"]})
```

Probar antes de ejecutar a escala

```
pytest -q tests/           # pruebas unitarias
snakemake --lint           # buenas practicas del flujo
snakemake -n --configfile tests/config_mini.yaml # ensayo en seco
snakemake --cores 2 --sdm conda --configfile tests/config_mini.yaml
```

Si esas cuatro órdenes se ejecutan automáticamente en cada *commit* (integración continua, por ejemplo con GitHub Actions), un error se detecta minutos después de introducirlo, y no semanas después, cuando un resultado no cuadra.

18.2.5 Documentación e informe reproducible

La documentación mínima de un proyecto es un README que responda cuatro preguntas: qué hace el proyecto, cómo se instala el entorno, cómo se ejecuta (idealmente una orden) y cómo se cita. A él se suman una licencia (sin licencia, legalmente nadie puede reutilizar el código), un archivo CITATION.cff y, para el código publicado, un DOI de archivo permanente.

El *informe* es el último eslabón del flujo, no un documento aparte. Un cuaderno Jupyter que lee las tablas de resultados/ y produce todas las figuras es una regla más, y debe poder ejecutarse de principio a fin sin intervención. Rule et al. (2019)

dan diez reglas para que un cuaderno sea legible y reutilizable, entre ellas contar una historia para una audiencia concreta, documentar el proceso y no solo los resultados, dividir el código en celdas con un propósito claro, registrar las dependencias y construir el cuaderno como parte de un flujo. MultiQC (Ewels et al., 2016) reúne en un solo informe HTML las métricas de calidad de todas las herramientas y muestras, y Snakemake puede generar un informe con la procedencia de cada resultado (`snakemake --report`). La regla 9 de Sandve et al. (2013), conectar cada afirmación del texto con el resultado que la sustenta, se cumple de forma natural si cada número del informe se lee de un archivo generado por el flujo en lugar de copiarse a mano.

18.2.6 Metadatos y publicación de datos

Las lecturas crudas de cualquier estudio publicado deben depositarse en un archivo público del consorcio INSDC: el *Sequence Read Archive* (SRA) del NCBI (Leinonen, Sugawara & Shumway, 2011) o el *European Nucleotide Archive* (ENA) del EMBL-EBI (Leinonen, Akhtar et al., 2011), que se replican entre sí. Ya los consultamos en el ??; ahora nos toca depositar. Ambos organizan los metadatos en una jerarquía (figura 18.7) que refleja la estructura de un experimento: un *proyecto* agrupa *muestras* biológicas; de cada muestra se preparan uno o varios *experimentos* (bibliotecas secuenciadas con una tecnología); cada experimento produce una o varias *corridas*, que contienen los archivos.

Los metadatos son la diferencia entre unos datos reutilizables y unos bytes huérfanos. Para la opción A, la fecha y el país de colecta, el hospedero y el tipo de muestra son imprescindibles para cualquier análisis epidemiológico posterior; para la opción B, la condición, la réplica, el tejido y el lote de preparación de bibliotecas. Los depósitos exigen *listas de verificación* (*checklists*) con campos obligatorios y vocabularios controlados precisamente para que los principios FAIR se cumplan en la práctica (Wilkinson et al., 2016). El archivo `config/muestras.tsv` del proyecto debería contener ya esas columnas, de modo que la misma tabla alimente el flujo y el depósito.

Las hojas de cálculo reescriben sus datos

Abrir una tabla de metadatos o de genes en una hoja de cálculo es arriesgado: los programas convierten silenciosamente nombres como *SEPT2* o *MARCH1* en fechas, e identificadores largos en notación científica. Ziemann et al. (2016) encontraron este tipo de error en aproximadamente una de cada

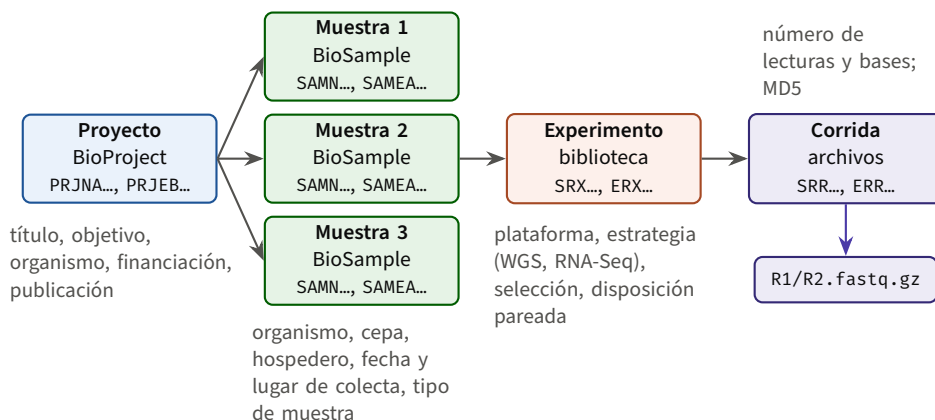


Figura 18.7. La jerarquía de objetos del SRA y el ENA, con los prefijos de sus accesiones (NCBI / EBI) y los metadatos que se describen en cada nivel. Un proyecto contiene muestras; cada muestra puede secuenciarse en varios experimentos (bibliotecas), y cada experimento en varias corridas. Las muestras llevan la información biológica que hará reutilizables los datos; los experimentos, la técnica. El artículo debe citar la accesión del proyecto, y el flujo debe descargar las lecturas por la accesión de cada corrida.

cinco publicaciones que incluían listas de genes en archivos de Excel como material suplementario. Edite los metadatos como texto plano (TSV), válidelos con un *script* y nunca guarde una tabla “pasada” por una hoja de cálculo como fuente de verdad.

18.2.7 Ética y datos humanos

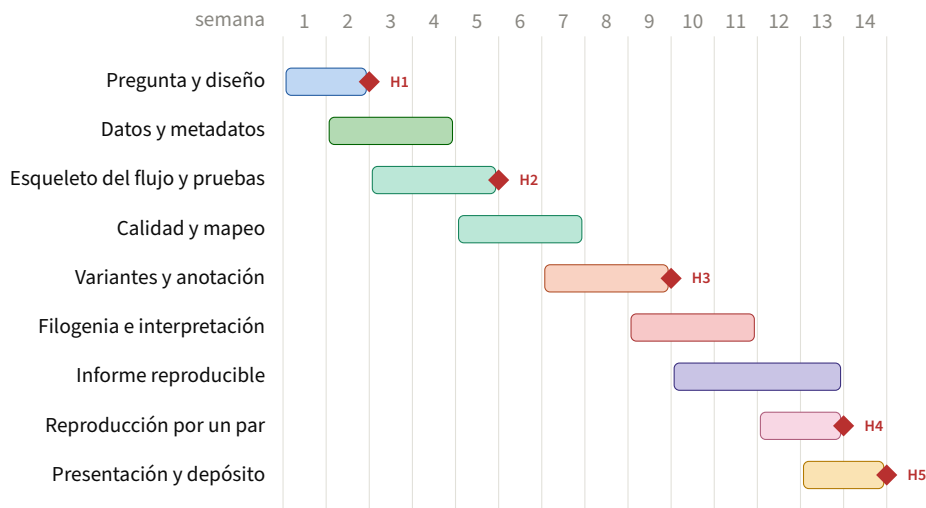
La opción A trabaja con genomas de bacterias, pero las muestras vienen de pacientes; la opción B puede usar ARN humano. En ambos casos entran en juego obligaciones éticas y legales que deben planificarse desde el diseño, no resolverse en el momento de publicar.

- **Consentimiento y aprobación ética.** El uso de datos humanos requiere la aprobación de un comité de ética y un consentimiento informado cuyo alcance cubra el uso previsto, incluido el depósito en repositorios públicos o de acceso controlado. Reutilizar datos públicos no exime de respetar las condiciones con que se recogieron.

- **La anonimización genómica es frágil.** Gymrek et al. (2013) demostraron que es posible inferir el apellido de participantes supuestamente anónimos a partir de los marcadores STR de su cromosoma Y, cruzándolos con bases de datos genealógicas recreativas, y, combinándolo con la edad y el lugar de residencia, reidentificarlos. Un genoma humano es, en la práctica, un identificador personal. Por eso los datos humanos individuales se depositan en archivos de *acceso controlado* (como dbGaP o el *European Genome-phenome Archive*), donde un comité evalúa cada solicitud.
- **Protección de datos.** En la Unión Europea, el Reglamento General de Protección de Datos (GDPR, aplicable desde mayo de 2018) clasifica los datos genéticos como una *categoría especial* de datos personales, cuyo tratamiento solo es lícito bajo condiciones específicas, como el consentimiento explícito o las excepciones para investigación con salvaguardas adecuadas, entre ellas la seudonimización; los datos seudonimizados siguen siendo datos personales (Shabani & Borry, 2018). Otros países tienen leyes propias (en Colombia, por ejemplo, la Ley 1581 de 2012 de protección de datos personales); compruebe cuál se aplica a sus datos y a su institución.
- **Lecturas del hospedero.** Las lecturas de un patógeno secuenciado a partir de una muestra clínica contienen a menudo lecturas humanas. Antes de depositarlas, elimínelas (mapeando contra la referencia humana y descartando lo que coincida) y documente ese paso en el flujo.
- **Metadatos identificadores.** Una fecha exacta de colecta, un hospital y una edad pueden bastar para identificar a un paciente en una comunidad pequeña. Reduzca la resolución (mes en lugar de día, región en lugar de dirección) cuando no afecte a la pregunta científica.

18.2.8 Cronograma

Un proyecto de este tamaño cabe en un semestre si se planifica con cuidado. Un cronograma es, de nuevo, un grafo acíclico dirigido de tareas con duraciones: las mismas redes PERT que Kahn (1962) quería ordenar. Su camino crítico indica qué retrasos retrasan la entrega final y cuáles tienen holgura. La figura 18.8 propone una distribución en 14 semanas con cinco hitos verificables.



Hitos: H1 propuesta aprobada (pregunta, datos, rúbrica); H2 el flujo corre de principio a fin sobre los datos mínimos de prueba; H3 resultados preliminares con todas las muestras; H4 un compañero reproduce el análisis desde el repositorio; H5 entrega: informe, repositorio etiquetado y presentación.

Figura 18.8. Cronograma orientativo del proyecto integrador en 14 semanas. Las tareas se solapan porque no todas dependen unas de otras: el esqueleto del flujo (con datos de prueba) se escribe mientras se descargan y curan los datos reales, y el informe se empieza a redactar en cuanto hay resultados preliminares. El camino crítico recorre diseño → esqueleto → mapeo → variantes → filogenia → informe: un retraso en cualquiera de ellos retrasa la entrega. El hito H4, en el que otra persona reproduce su análisis, es la prueba de fuego del capítulo; resérvele tiempo.

18.2.9 Rúbrica de evaluación

La rúbrica hace explícito lo que se espera y permite que la autoevaluación coincida con la evaluación. La tabla 18.3 propone seis criterios, cada uno puntuado en una escala de 1 (insuficiente) a 4 (excelente), con pesos w_i que suman 1. La nota final es la media ponderada

$$N = \sum_{i=1}^6 w_i x_i, \quad \sum_{i=1}^6 w_i = 1, \quad x_i \in \{1, 2, 3, 4\}, \quad (18.14)$$

que se reescala a la escala de la institución.

Tabla 18.3. Rúbrica del proyecto integrador. Cada criterio se puntúa de 1 a 4; se describen los extremos de la escala.

Criterio	w_i	Nivel 4 (excelente)	Nivel 1 (insuficiente)
Pregunta y diseño	0,20	pregunta precisa; datos adecuados y justificados; limitaciones reconocidas	pregunta vaga; datos elegidos por conveniencia
Corrección del análisis	0,20	métodos apropiados, parámetros justificados, controles de calidad interpretados	errores metodológicos; salidas sin inspeccionar
Reproducibilidad	0,15	un par reproduce todo con una orden; entornos y datos fijados	pasos manuales; versiones desconocidas
Calidad del código y pruebas	0,15	flujo legible, modular, con pruebas unitarias y de integración	<i>script</i> monolítico, sin pruebas
Datos, metadatos y ética	0,15	metadatos completos; depósito o plan de depósito; ética atendida	metadatos ausentes; ética ignorada
Informe y comunicación	0,15	figuras claras generadas por el flujo; conclusiones apoyadas en resultados	figuras copiadas a mano; conclusiones sin sustento

DONDE

w_i	Peso del criterio i en la nota final.
x_i	Nivel alcanzado en el criterio i .
N	Nota ponderada, entre 1 y 4.

Ejemplo 18.4. (Aplicar la rúbrica)

Un proyecto obtiene los niveles $x = (4, 3, 3, 4, 2, 3)$ en los seis criterios, en el orden de la tabla 18.3. Su nota es

$$\begin{aligned}
 N &= 0,20 \cdot 4 + 0,20 \cdot 3 + 0,15 \cdot 3 + 0,15 \cdot 4 + 0,15 \cdot 2 + 0,15 \cdot 3 \\
 &= 0,80 + 0,60 + 0,45 + 0,60 + 0,30 + 0,45 = 3,20,
 \end{aligned}$$

es decir, un 80 % de la nota máxima (4,0 sobre 5). El punto débil está en datos, metadatos y ética ($x_5 = 2$): subirlo a 4, completando la tabla de metadatos y el plan de depósito, añadiría $0,15 \times 2 = 0,30$ puntos y llevaría la nota a 3,50 (87,5 %). La rúbrica no solo califica: indica dónde rinde más el esfuerzo que queda.

El proyecto está terminado cuando otra persona, con su repositorio, las accesiones de los datos y una sola orden, obtiene las mismas tablas y figuras que usted.

En resumen

- Un análisis es un grafo acíclico dirigido de trabajos. Existe un orden de ejecución válido si y solo si no hay ciclos, y el algoritmo de Kahn lo encuentra, o detecta el ciclo, en tiempo $O(|V| + |E|)$ manteniendo los grados de entrada residuales.
- `make` introdujo las dependencias por archivos y el criterio de rehacer lo desactualizado. `Snakemake` lo lleva a Python con comodines e inferencia del grafo hacia atrás (*pull*); `Nextflow` usa procesos aislados comunicados por canales de cola y de valor en un modelo de flujo de datos (*push*); `CWL` es un estándar declarativo multimotor.
- La reproducibilidad exige fijar código (Git), entorno (Bioconda, contenedores con *digest*), datos (accesiones y SHA-256), parámetros (configuración versionada) y semillas. Las claves de caché por *hash* permiten reanudar un flujo rehaciendo solo los descendientes de lo que cambió.
- El tiempo en paralelo está acotado por $\max(W/P, S) \leq T_p \leq W/P + S$; la ley de Amdahl limita la aceleración a $1/s$ y la de Gustafson muestra que los problemas más grandes escalan mejor. Acortar el camino crítico (p. ej., dividiendo el llamado conjunto por regiones) rinde más que añadir núcleos.
- Un proyecto integrador reproducible tiene una estructura de directorios clara, control de versiones, pruebas a varios niveles, un informe generado por el propio flujo, datos depositados con metadatos FAIR y un tratamiento ético y legal de los datos humanos planificado desde el diseño.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 18.1** ●●● Dibuje el grafo de trabajos del Snakefile con dos muestras en lugar de tres. ¿Cuántos trabajos y aristas tiene? Compruebe que la suma de los grados de entrada es igual al número de aristas.
- 18.2** ●●● Ejecute a mano el algoritmo de Kahn sobre el grafo del ejercicio anterior con una *pila* (LIFO) en lugar de una cola. ¿Obtiene el mismo orden? ¿Es igualmente válido? ¿Por qué el teorema de corrección no depende de la estructura de datos elegida?
- 18.3** ●●● Con la ecuación (18.3), determine qué trabajos del flujo de la figura 18.1 se rehacen si se actualiza el archivo de la referencia. Compare con el caso en que solo cambia la base de datos de SnpEff.
- 18.4** ●●● Añada al Snakefile una regla `mosdepth` que calcule la cobertura de cada muestra (??) y haga que `multiqc` la incluya. ¿Qué aristas nuevas aparecen en el grafo? ¿Cambia el camino crítico?
- 18.5** ●●● Escriba la versión Nextflow de la regla del ejercicio anterior. Explique, usando la ecuación (18.5), qué pasaría si conectara el proceso a la salida de DEDUP y a un canal de cola que contiene solo la referencia.
- 18.6** ●●● Implemente en Python el cálculo del camino crítico (ecuación (18.7)) sobre el orden que devuelve `orden_topologico`, y verifique que con las duraciones del ejemplo 18.2 obtiene $S = 46$ min para tres muestras.
- 18.7** ●●● Mida la aceleración de un flujo real (por ejemplo, el Snakefile con datos de prueba) con 1, 2, 4 y 8 núcleos, calcule la fracción serial de Karp-Flatt (ecuación (18.11)) y diga si el límite es una parte serial genuina o una sobrecarga.
- 18.8** ●●● Demuestre que el número de rondas de la versión por niveles del algoritmo de Kahn es igual al número de vértices del camino más largo del grafo, y que el ancho máximo de una ronda es una cota inferior del número de núcleos necesarios para alcanzar $T_p = S$ cuando todas las duraciones son iguales. ¿Es también una cota superior?

- 18.9** ●●● Deduzca la ecuación (18.13) y calcule cuántos archivos harían falta para que la probabilidad de una colisión accidental de SHA-256 alcanzara 10^{-6} . Compare con el número estimado de archivos del SRA.
- 18.10** ●●● Diseñe su propio proyecto integrador: escriba la pregunta, elija las accesiones de los datos, dibuje el grafo de trabajos, estime el presupuesto de disco y cómputo como en el ejemplo 18.3, redacte la sección de ética y autoevalúe el diseño con la rúbrica de la tabla 18.3.

Referencias del capítulo

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67)*, 483–485. <https://doi.org/10.1145/1465482.1465560>
- Buckheit, J. B., & Donoho, D. L. (1995). WaveLab and Reproducible Research. En A. Antoniadis & G. Oppenheim (Eds.), *Wavelets and Statistics* (pp. 55–81). Springer. https://doi.org/10.1007/978-1-4612-2544-7_5
- Crusoe, M. R., Abeln, S., Iosup, A., Amstutz, P., Chilton, J., Tijanić, N., Ménager, H., Soiland-Reyes, S., Gavrilović, B., Goble, C., & The CWL Community. (2022). Methods included: Standardizing computational reuse and portability with the Common Workflow Language. *Communications of the ACM*, 65(6), 54–63. <https://doi.org/10.1145/3486897>
- da Veiga Leprevost, F., Grüning, B. A., Alves Aflitos, S., Röst, H. L., Uszkoreit, J., Barsnes, H., Vaudel, M., Moreno, P., Gatto, L., Weber, J., Bai, M., Jimenez, R. C., Sachsenberg, T., Pfeuffer, J., Vera Alvarez, R., Griss, J., Nesvizhskii, A. I., & Perez-Riverol, Y. (2017). BioContainers: An open-source and community-driven framework for software standardization. *Bioinformatics*, 33(16), 2580–2582. <https://doi.org/10.1093/bioinformatics/btx192>
- Di Tommaso, P., Chatzou, M., Floden, E. W., Barja, P. P., Palumbo, E., & Notredame, C. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4), 316–319. <https://doi.org/10.1038/nbt.3820>
- Ewels, P., Magnusson, M., Lundin, S., & Käller, M. (2016). MultiQC: Summarize analysis results for multiple tools and samples in a single report. *Bioinformatics*, 32(19), 3047–3048. <https://doi.org/10.1093/bioinformatics/btw354>
- Ewels, P. A., Peltzer, A., Fillinger, S., Patel, H., Alneberg, J., Wilm, A., Garcia, M. U., Di Tommaso, P., & Nahnsen, S. (2020). The nf-core framework for community-curated bioinformatics pipelines. *Nature Biotechnology*, 38(3), 276–278. <https://doi.org/10.1038/s41587-020-0439-x>
- Feldman, S. I. (1979). Make—a program for maintaining computer programs. *Software: Practice and Experience*, 9(4), 255–265. <https://doi.org/10.1002/spe.4380090402>

- Grüning, B., Chilton, J., Köster, J., Dale, R., Soranzo, N., van den Beek, M., Goecks, J., Backofen, R., Nekrutenko, A., & Taylor, J. (2018). Practical computational reproducibility in the life sciences. *Cell Systems*, 6(6), 631-635. <https://doi.org/10.1016/j.cels.2018.03.014>
- Gustafson, J. L. (1988). Reevaluating Amdahl's law. *Communications of the ACM*, 31(5), 532-533. <https://doi.org/10.1145/42411.42415>
- Gymrek, M., McGuire, A. L., Golan, D., Halperin, E., & Erlich, Y. (2013). Identifying personal genomes by surname inference. *Science*, 339(6117), 321-324. <https://doi.org/10.1126/science.1229566>
- Hadfield, J., Megill, C., Bell, S. M., Huddleston, J., Potter, B., Callender, C., Sagulenko, P., Bedford, T., & Neher, R. A. (2018). Nextstrain: Real-time tracking of pathogen evolution. *Bioinformatics*, 34(23), 4121-4123. <https://doi.org/10.1093/bioinformatics/bty407>
- Kahn, A. B. (1962). Topological sorting of large networks. *Communications of the ACM*, 5(11), 558-562. <https://doi.org/10.1145/368996.369025>
- Karp, A. H., & Flatt, H. P. (1990). Measuring parallel processor performance. *Communications of the ACM*, 33(5), 539-543. <https://doi.org/10.1145/78607.78614>
- Köster, J., & Rahmann, S. (2012). Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19), 2520-2522. <https://doi.org/10.1093/bioinformatics/bts480>
- Kurtzer, G. M., Sochat, V., & Bauer, M. W. (2017). Singularity: Scientific containers for mobility of compute. *PLOS ONE*, 12(5), e0177459. <https://doi.org/10.1371/journal.pone.0177459>
- Leinonen, R., Akhtar, R., Birney, E., Bower, L., Cerdeno-Tarraga, A., Cheng, Y., Cleland, I., Faruque, N., Goodgame, N., Gibson, R., Hoad, G., Jang, M., Pakseresht, N., Plaister, S., Radhakrishnan, R., Reddy, K., Sobhany, S., Ten Hoopen, P., Vaughan, R., Zalunin, V., et al. (2011). The European Nucleotide Archive. *Nucleic Acids Research*, 39(Database), D28-D31. <https://doi.org/10.1093/nar/gkq967>
- Leinonen, R., Sugawara, H., & Shumway, M. (2011). The Sequence Read Archive. *Nucleic Acids Research*, 39(Database), D19-D21. <https://doi.org/10.1093/nar/gkq1019>
- Mölder, F., Jablonski, K. P., Letcher, B., Hall, M. B., Tomkins-Tinch, C. H., Sochat, V., Forster, J., Lee, S., Twardziok, S. O., Kanitz, A., Wilm, A., Holtgrewe, M., Rahmann, S., Nahnsen, S., & Köster, J. (2021). Sustainable data analysis with Snakemake. *F1000Research*, 10, 33. <https://doi.org/10.12688/f1000research.29032.2>
- Noble, W. S. (2009). A quick guide to organizing computational biology projects. *PLoS Computational Biology*, 5(7), e1000424. <https://doi.org/10.1371/journal.pcbi.1000424>
- Peng, R. D. (2011). Reproducible research in computational science. *Science*, 334(6060), 1226-1227. <https://doi.org/10.1126/science.1213847>
- Perez-Riverol, Y., Gatto, L., Wang, R., Sachsenberg, T., Uszkoreit, J., Leprevost, F. d. V., Fufezan, C., Ternent, T., Eglen, S. J., Katz, D. S., Pollard, T. J., Kononov, A., Flight, R. M., Blin, K., & Vizcaíno, J. A. (2016). Ten simple rules for taking advantage of Git and GitHub. *PLOS Computational Biology*, 12(7), e1004947. <https://doi.org/10.1371/journal.pcbi.1004947>

- Rule, A., Birmingham, A., Zuniga, C., Altintas, I., Huang, S.-C., Knight, R., Moshiri, N., Nguyen, M. H., Rosenthal, S. B., Pérez, F., & Rose, P. W. (2019). Ten simple rules for writing and sharing computational analyses in Jupyter Notebooks. *PLOS Computational Biology*, 15(7), e1007007. <https://doi.org/10.1371/journal.pcbi.1007007>
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10), e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>
- Shabani, M., & Borry, P. (2018). Rules for processing genetic data for research purposes in view of the new EU General Data Protection Regulation. *European Journal of Human Genetics*, 26(2), 149-156. <https://doi.org/10.1038/s41431-017-0045-7>
- The Bioconda Team, Grüning, B., Dale, R., Sjödin, A., Chapman, B. A., Rowe, J., Tomkins-Tinch, C. H., Valieris, R., & Köster, J. (2018). Bioconda: Sustainable and comprehensive software distribution for the life sciences. *Nature Methods*, 15(7), 475-476. <https://doi.org/10.1038/s41592-018-0046-7>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., Gonzalez-Beltran, A., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>
- Wratten, L., Wilm, A., & Göke, J. (2021). Reproducible, scalable, and shareable analysis pipelines with bioinformatics workflow managers. *Nature Methods*, 18(10), 1161-1168. <https://doi.org/10.1038/s41592-021-01254-9>
- Ziemann, M., Eren, Y., & El-Osta, A. (2016). Gene name errors are widespread in the scientific literature. *Genome Biology*, 17(1), 177. <https://doi.org/10.1186/s13059-016-1044-7>